Technical Handbook

Version 1.1 · July 18, 2026

Autor	Maximilian Jakob Maag B.Sc.
Audience	Administrators, Developers, Project Managers
System	Infrastructure Webshop
Stack	Next.js · TypeScript · PostgreSQL · OpenTofu · GitLab
Scope	Architecture · Operations · Development · Product Lifecycle

Open Hybrid Cloud — Digital Unit Infrastructure Team

Contents

List of Figures

List of Tables

Listings

Part I

Overview & Architecture

Chapter 1

Introduction

1.1 Purpose of This Handbook

Open Hybrid Cloud is an open-source, self-service portal that enables Admins and Project Managers to order, manage, and decommission IT infrastructure on Linode/Akamai Cloud through a browser-based interface. The source code is publicly available at https://github.com/Maximilian-Maag/open-hybrid-cloud. This handbook is the authoritative reference for everyone who operates, extends, or uses the system.

It covers:

- The complete system architecture described with the C4 model
- Infrastructure-as-Code modules and the GitLab CI/CD concept
- Architecture and design decisions with their rationale
- Step-by-step administrator operations
- Developer onboarding, code structure, and extension patterns
- The full product lifecycle from creation to decommissioning

1.2 Target Audiences

Admin Responsible for approving orders, monitoring all infrastructure, and ordering directly without an approval step. Reads Part III and Part V.

Root Responsible for the product catalog, system configuration, and local user accounts. Reads Part III entirely.

Developer Extends or maintains the webshop codebase and the OpenTofu modules. Reads Part I and Part IV.

Project Manager Places infrastructure orders and manages projects. Reads Part V.

1.3 Conventions Used in This Handbook

`monospace` File paths, commands, code, environment variables, and route patterns.

italic Technical terms on first use.

bold UI labels, button names, and menu items.

Note boxes Yellow-highlighted boxes providing additional context or important background information.

Important boxes Orange-highlighted boxes warning about actions that can cause data loss or service disruption.

Tip boxes Green-highlighted boxes with shortcuts and best practices.

1.4 System At a Glance

The webshop is a Next.js monorepo with a dedicated backend API and a React-based frontend. Orders are fulfilled by triggering CI/CD pipelines (GitLab, GitHub, or Bitbucket) that run OpenTofu (a Terraform-compatible open-source tool) to provision or destroy cloud resources on Linode/Akamai. Pipeline status is reported back via webhooks rather than polling.

Table 1.1: Technology stack overview

Layer	Technology	Version
Runtime	Node.js	22+
Backend	Next.js 15 (API-only)	15.x
Frontend	Next.js 15 + React	15.x / 19.x
CSS	Tailwind CSS v4	4.x
Package Mgr	pnpm (workspace)	9.x
ORM	Drizzle ORM	—
Database	PostgreSQL	15+
Auth (backend)	JWT HS256 (jose)	24 h expiry
Auth (frontend)	NextAuth v5	5.x
IaC	OpenTofu	1.9+
Cloud	Linode/Akamai	—
CI/CD	GitLab / GitHub / Bitbucket	16+ / — / —
State Backend	GitLab Managed TF State	—

Chapter 2

C4 Architecture Model

The system is described using the *C4 model* (Context, Container, Component, Code). Each level zooms in one step deeper.

2.1 Level 1 — System Context

Figure 2.1 shows who uses the webshop and which external systems it depends on.

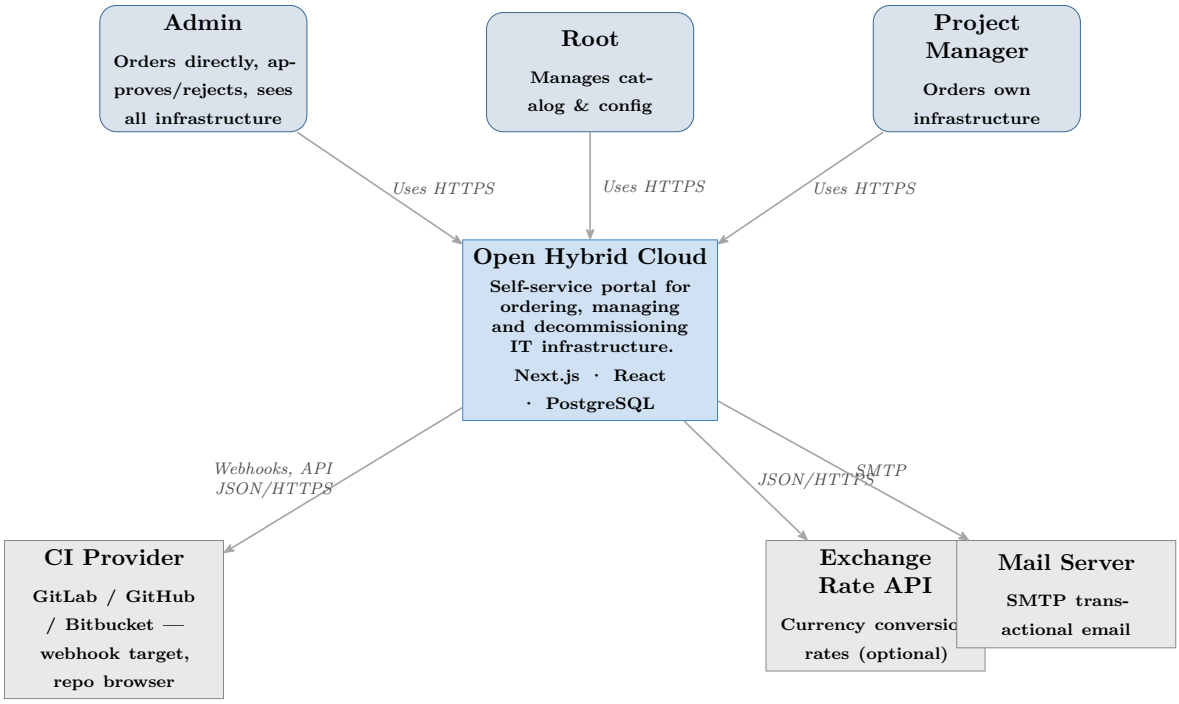

Figure 2.1: C4 Level 1 — System Context Diagram

2.1.1 Actors

Admin Authenticates via local account with JWT. Has the system role `admin`. Can order infrastructure directly (no approval required), approve or reject orders from

Project Managers, view all projects and infrastructure, and decommission any resource.

Root Uses a local account. Has the system role `root`. Manages the product catalog, system configuration (SMTP, branding, currencies), environments, CI sources, cost centers, and local users. Cannot place orders.

Project Manager Authenticates via local account with JWT. Has the system role `project_manager`. Places orders (which require Admin approval), manages their own projects, and can decommission their own infrastructure.

2.1.2 External Systems

CI Provider (GitLab / GitHub / Bitbucket) One or more self-hosted or cloud CI provider instances. The webshop connects for two purposes: (1) browsing repositories and importing `variables.tf` parameter definitions via the GitLab API, and (2) triggering provisioning and decommissioning webhooks. Pipeline status is reported back to the webshop via inbound webhook events (`POST /api/webhooks/{provider}/pipeline`).

Exchange Rate API Optional. Any HTTP JSON exchange rate endpoint. Used to refresh currency conversion rates. Configured via `EXCHANGE_RATE_API_URL` and `EXCHANGE_RATE_API_KEY`.

Mail Server Any SMTP server. Sends order confirmations, approval request notifications, approval/rejection notices, deployment completion alerts, and failure notifications. Delivered via nodemailer.

2.2 Level 2 — Container Diagram

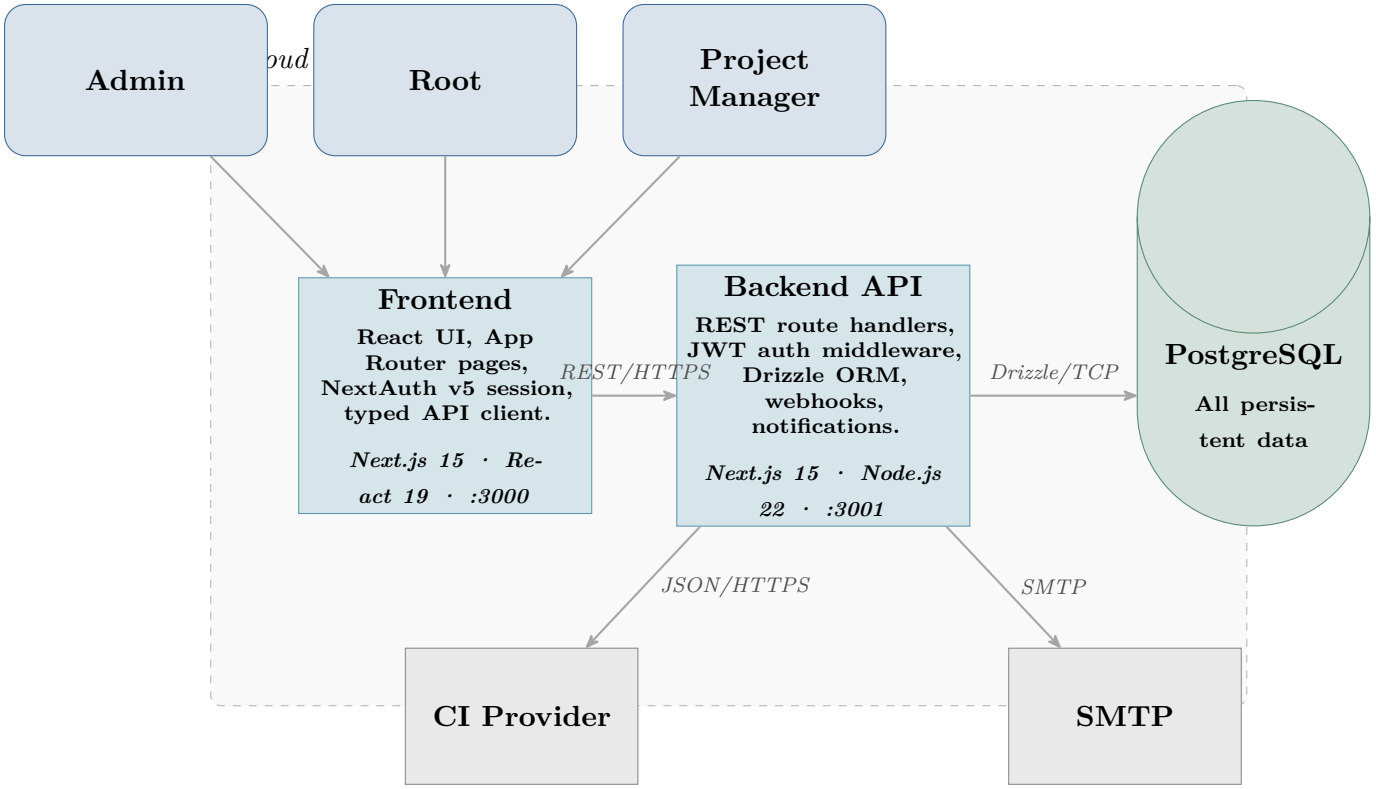

Figure 2.2: C4 Level 2 — Container Diagram

The system has three containers (plus the shared database):

Frontend (port 3000) A Next.js 15 application serving the React UI via the App Router. All pages are React Server Components or client components. It communicates with the backend exclusively via typed fetch wrappers in `src/lib/api.ts`. Authentication state is managed by NextAuth v5, which stores the JWT received from the backend in a secure server-side session.

Backend API (port 3001) A Next.js 15 application running in API-only mode — no UI pages. Route handlers under `src/app/api/` are thin shells: each one checks authentication, parses and validates the request body with Zod, delegates to a typed service function in `src/lib/services/`, and maps the returned `Result<T>` to a `NextResponse` via `toResponse()`. All business logic, state machine transitions, CI orchestration, email notifications, and audit writes live in the service layer, never in route handlers. JWT middleware (`src/lib/auth/`) verifies every authenticated request. Drizzle ORM manages all database access. The webhook handler (`src/lib/webhook/`) processes inbound pipeline status events from CI providers.

PostgreSQL Database The single source of truth. Stores the product catalog, orders, infrastructure state, audit log, branding, configuration, users, and exchange rates. The database is the only stateful component.

2.3 Level 3 — Component Diagram

Figure 2.3 decomposes the backend API container into its major logical components.

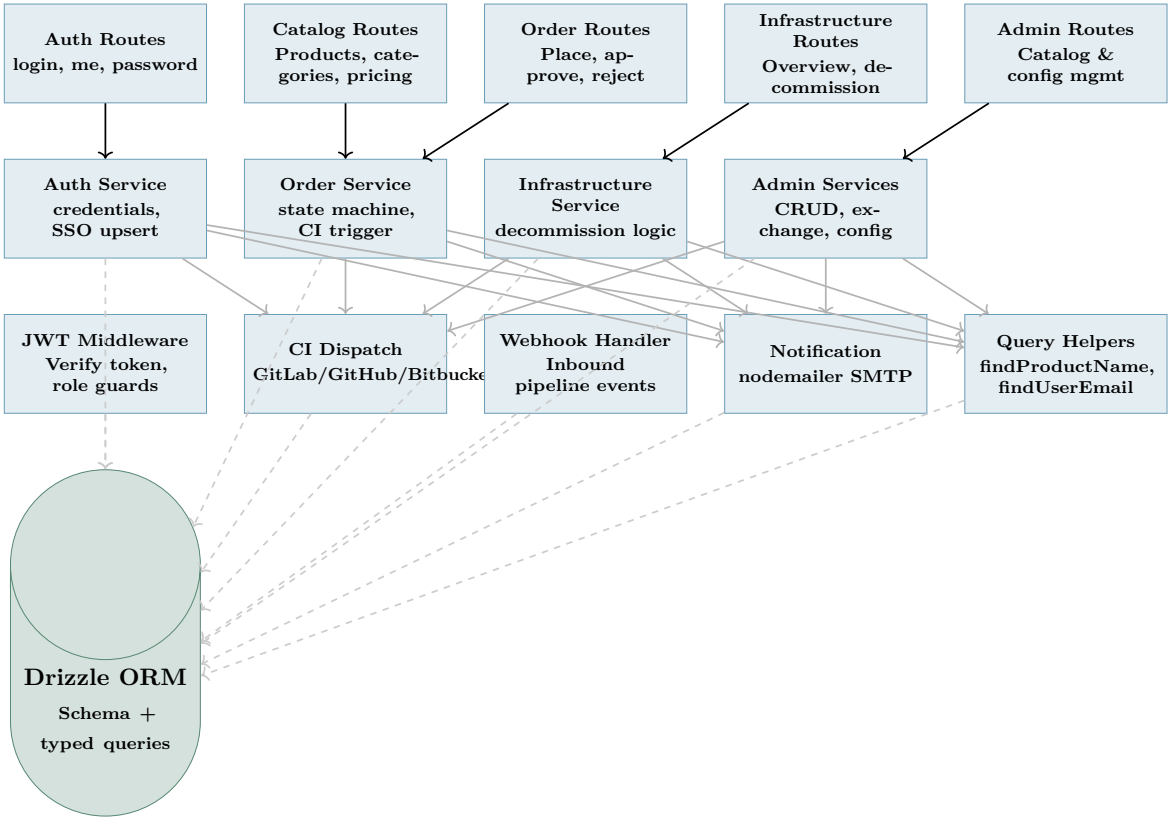

Figure 2.3: C4 Level 3 — Component Diagram (Backend API)

2.3.1 Component Descriptions

Route handlers are intentionally thin: they perform authentication, validate the request with Zod, call a service function, and return a `NextResponse` via `toResponse()`. All domain logic is in the service layer.

Component	Responsibility
Route Handlers (`src/app/api/`)	
Auth Routes	Parse credentials or OIDC callback, call the auth service, return a signed JWT. No business logic inline.
Catalog Routes	Parse language/filter query params, call the catalog service.
Order Routes	Validate `CreateOrderSchema` / `RejectSchema`, call the orders or approvals service, return the result.
Infrastructure Routes	Role-scoped list and decommission trigger; delegates entirely to the infrastructure service.

Component	Responsibility
Admin Routes	Thin CRUD shells for catalog, environments, CI sources, users, config, and branding — each delegates to the corresponding admin service.
Service Layer (src/lib/services/)	
Auth Service	Verifies bcrypt passwords, upserts SSO users, returns or invalidates JWTs. Returns `Result<string>` (token) or a typed error.
Order / Approval Services	Implements the order state machine: *pending* → *provisioning* → *completed* / *failed* / *rejected*. Calls `triggerProductWebhooks`, writes the audit log, and dispatches email notifications.
Infrastructure Service	Ownership check for project managers, status guard (*active* only), CI destroy trigger, audit write.
Admin Services	One service file per domain (`users`, `products`, `categories`, `environments`, `ciSources`, `parameters`, `costCenters`, `exchangeRates`, `config`, `branding`). Each returns typed `Result<T>` values.
Shared Libraries (src/lib/)	
Result Type	`lib/services/result.ts` — `Ok<T>` / `Err` discriminated union with `ok()` and `err(status, message)` constructors. `lib/http.ts` maps any `Result<T>` to a `NextResponse` via `toResponse()`.
Query Helpers	`lib/db/queries.ts` — shared read helpers used across multiple services: `findProductName`, `findUserEmail`, `findAdminEmails`, `findCiSourceForEnv`. Eliminates repeated join patterns.
JWT Middleware	Verifies Bearer tokens on every authenticated route using the `jose` library. Exports typed role guard wrappers (`requireRole`, `requireAuth`).
CI Dispatch	Provider-agnostic module in `src/lib/ci/` that triggers pipelines via GitLab, GitHub, or Bitbucket APIs. `lib/ci/webhooks.ts` exposes two trigger functions: `triggerProductWebhooks()` — fetches per-product webhooks in `exec_order` and calls each; and `triggerPipelineStacks()` — fetches pipeline stacks for the product+environment, serialises the ordered step list as `PIPELINE_STACK` JSON, and triggers the configured orchestrator pipeline. Both return pipeline IDs that are stored on the order.

Component	Responsibility
Webhook Handler	Inbound handler at `POST /api/webhooks/{provider}/pipeline`. Maps CI pipeline status events to order/infrastructure state updates; fetches job traces to parse OpenTofu outputs on success.
Notification	Wraps nodemailer in `src/lib/notification/`. Seven typed send functions; all user-controlled strings are HTML-escaped before embedding. Silently no-ops when SMTP is not configured.
Bootstrap	On startup, seeds the initial Root account from `ADMIN_EMAIL` / `ADMIN_PASSWORD` if no users exist.
Drizzle ORM	Schema definitions and typed query builders in `src/lib/db/`. All database access goes through Drizzle.

Table 2.1: Component descriptions

2.4 Level 4 — Key Code Flows

2.4.1 Order Placement Flow (Project Manager)

1. Frontend calls `POST /api/orders` with JWT in the `Authorization` header.
2. The route handler verifies the JWT via middleware and validates the request body with Zod's `CreateOrderSchema`.
3. The handler calls `createOrder(session, input)` in `lib/services/orders.ts`. The service inserts an order with status `pending`, writes an audit entry, sends an order-created email to the orderer, and sends approval-request emails to all active Admins.
4. `createOrder` returns `Ok<CreatedOrder>`; `toResponse` serialises it as HTTP 201.
5. The frontend navigates to the order detail page showing *Awaiting Approval*.

2.4.2 Order Approval & Provisioning Flow

1. Admin calls `POST /api/approvals/:id/approve`.
2. The route handler validates auth and delegates to `approveOrder(session, orderId)` in `lib/services/approvals.ts`.
3. The service fetches the order (returns `Err 400` if not `pending`), then calls `triggerProductWebhooks()` and `triggerPipelineStacks()` in `lib/ci/webhooks.ts` in parallel. Product webhooks fire in `exec_order`; pipeline stacks serialise the configured step sequence as `PIPELINE_STACK` JSON and POST it to each stack's orchestrator webhook URL.

4. All returned pipeline IDs are merged and stored in the `pipeline_id` JSONB column; an infra element is created; an audit row is written; the orderer receives an approval email.
5. When the CI pipeline completes, the provider calls `POST /api/webhooks/{provider}/pipeline`. The webhook handler in `lib/webhook/handler.ts` transitions the order to `completed`, parses OpenTofu outputs from the job trace, and sends a provisioning-complete email.
6. On pipeline failure the order transitions to `failed` and a failure email is sent to the orderer.

2.4.3 Request–Response Lifecycle (Next.js)

Every backend request follows the same four-step shell:

1. **Auth** — `requireAuth(req)` or `requireRole('admin')(req)` verifies the Bearer JWT and returns a typed `SessionUser`.
2. **Parse** — Zod `safeParse` validates the request body or query params; invalid input returns HTTP 400 immediately.
3. **Service** — the route handler calls the appropriate service function, which performs all DB work, CI calls, emails, and audit writes, and returns `Result<T>`.
4. **Respond** — `toResponse(result, successStatus)` converts `Ok<T>` to 200/201 JSON and `Err` to the appropriate HTTP error response.

User interactions on the frontend are handled by React Server Components or client components that call the backend via typed wrappers in `src/lib/api.ts`. NextAuth v5 stores the JWT in an encrypted server-side session cookie and forwards it as a Bearer token on every backend call.

Chapter 3

Design Decisions & Technology Rationale

3.1 Service Layer Architecture

Decision: All business logic is extracted from Next.js route handlers into typed service functions under `src/lib/services/`. Route handlers are restricted to three responsibilities: authentication check, Zod validation, and returning a `NextResponse` via `toResponse()`.

Rationale:

- **Testability:** Service functions take plain typed inputs and return `Result<T>` values. They can be tested directly without constructing `NextRequest` objects or mocking the HTTP layer.
- **Single responsibility:** Route handlers contain no `if/else` branches for business rules, no duplicate DB query patterns, and no notification calls. Every policy decision (state machine, ownership check, role guard) is in one place.
- **Shared infrastructure:** Repeated query patterns that previously appeared six or more times inline (*fetch product name*, *fetch user email*, *get CI source for environment*) are consolidated in `lib/db/queries.ts` and called from services and the webhook handler alike.
- **Explicit error paths:** The `Result<T>` discriminated union (`Ok<T>` | `Err`) makes every failure path visible at the call site. There are no silent `null` returns or exception-based control flow for business logic errors.

The pattern is: `ok(data)` for success, `err(httpStatus, message)` for domain errors (not found, forbidden, bad state). Database exceptions from Drizzle bubble up as unhandled rejections and are caught by Next.js's error boundary, returning a generic 500. This keeps error-handling code proportional to the actual surface that needs it.

3.2 Next.js Monorepo

Decision: The application is structured as a pnpm workspace monorepo with two Next.js applications (`apps/backend` and `apps/frontend`) and a shared types package (`packages/types`).

Rationale:

- TypeScript is shared end-to-end: the same interface definitions in `packages/types/` are consumed by both the backend route handlers and the frontend API client, making type drift a compile error rather than a runtime surprise.
- Separating the backend and frontend into distinct Next.js applications allows each to have an independent Dockerfile, independent environment variables, and an independent deploy cycle. The frontend can be updated without touching the API container.
- Next.js App Router (both apps) provides file-system-based routing, React Server Components, and built-in streaming without additional framework glue.
- pnpm workspaces deduplicate `node_modules` across packages, reducing install size and ensuring consistent package versions across the monorepo.

3.3 React 19 with App Router

Decision: The frontend is a Next.js 15 application using React 19 Server and Client Components via the App Router.

Rationale:

- **Type-safe API layer:** All backend calls are made through typed wrappers in `src/lib/api.ts`. The shared `packages/types` interfaces guarantee that the shape of every response is verified at compile time.
- **Server Components by default:** Data-fetching pages are React Server Components that call the backend on the server, avoiding client-side waterfall requests and keeping JWTs server-side.
- **Incremental interactivity:** Only the components that require event handlers or browser APIs are marked `"use client"`, minimising the client-side JavaScript bundle.

3.4 Tailwind CSS v4

Decision: Tailwind CSS v4 is used for all styling via the single `@import "tailwindcss"` directive. No DaisyUI. No separate build step.

Rationale:

- **Zero-config build:** Tailwind v4 uses a native CSS engine that requires only the single import; there is no `tailwind.config.js` or separate `build:css` command. Next.js handles the PostCSS pass automatically.
- **Design independence:** Pure utility classes keep every style decision explicit and auditable. Brand tokens (`var(-bp)`, `var(-bs)`) drive themed surfaces without conflicting with a component library's own custom properties.

3.5 PostgreSQL with Drizzle ORM

Decision: PostgreSQL 15+, accessed via the Drizzle ORM with the `postgres` Node.js driver.

Rationale:

- **Type-safe queries:** Drizzle generates TypeScript types from the schema definition in `src/lib/db/schema.ts`. Every query result is typed; shape mismatches are caught by the TypeScript compiler.
- **Schema as code:** `drizzle-kit push` applies the schema diff directly, removing the need for manually written SQL migration files while keeping the schema version-controlled.
- **Thin abstraction:** Drizzle stays close to SQL semantics (no opaque magic), making it straightforward to write efficient queries for JSONB columns, upserts, and aggregations.
- BYTEA columns store product images and the branding logo directly in the database, eliminating a separate object store at this scale.

3.6 Two-Container Architecture

Decision: The backend API and the frontend UI run in separate containers (`apps/backend/Dockerfile` and `apps/frontend/Dockerfile`).

Rationale: Separating concerns at the container boundary gives each part its own deployment, scaling, and environment-variable scope. The frontend never holds database credentials; the backend never renders HTML. Nginx (or any reverse proxy) routes `/api/*` to port 3001 and all other paths to port 3000. At the anticipated load (tens of users, hundreds of orders per month), this split adds minimal operational overhead while making the security boundary explicit.

3.7 OpenTofu over Terraform

Decision: All IaC modules use OpenTofu 1.9+.

Rationale: OpenTofu is a community-governed, truly open-source fork of Terraform created after HashiCorp changed Terraform's license to BSL 1.1 in 2023. The API, HCL

syntax, provider ecosystem, and state format are fully compatible. Using OpenTofu avoids future licensing costs and complies with Open Hybrid Cloud's open-source software policy.

3.8 GitLab Managed Terraform State

Decision: OpenTofu state is stored in GitLab's built-in HTTP-compatible state backend (GitLab Managed Terraform State).

Rationale:

- Reuses existing GitLab infrastructure — no S3 bucket, no Terraform Cloud account, no Vault setup required.
- State is protected by GitLab's RBAC: only project members with Developer or higher access can read or modify state.
- State locking via HTTP `LOCK`/`UNLOCK` prevents concurrent apply operations.
- State is browsable in the GitLab UI under **Infrastructure → Terraform states**.

3.9 Per-Resource Isolated State

Decision: Each VM and each Kubernetes cluster has its own isolated OpenTofu state (named `vm-{label}` and `k8s-{label}`).

Rationale: Shared state across multiple VMs creates blast-radius risk: a failing `apply` for one VM could lock the state and block unrelated VMs. Isolated states mean each resource's lifecycle is entirely independent. The trade-off is a larger number of state files, which is manageable at the expected scale.

3.10 Webhook-Driven Pipeline Trigger

Decision: The webshop triggers GitLab pipelines via webhook (HTTP POST) rather than the GitLab API's pipeline-trigger endpoint.

Rationale: Webhooks are the native GitLab mechanism for external triggers. They carry a secure token for authentication and include the ability to pass variables (OpenTofu parameters) as JSON. The pipeline configuration (`.gitlab-ci.yml`) lives in the same repository as the IaC code, making the trigger configuration auditable and version-controlled.

3.11 Pipeline Stacks — Portal-Defined CI DAGs

Decision: Introduce a `pipeline_stacks` table that stores an ordered list of CI/CD template steps per product+environment. On order approval the portal serialises the

step list as `PIPELINE_STACK` JSON and passes it to a generic orchestrator pipeline, eliminating the need to hard-code step sequences in `.gitlab-ci.yml`.

Rationale:

- **No CI YAML churn:** Adding or reordering infra steps for a product requires only a portal UI change — the GitLab orchestrator reads `PIPELINE_STACK` at runtime. A new step is live as soon as it is saved.
- **State key stability:** Each step appends a unique `stateSuffix` to the order parameter named by `stateKeyParam` (default: `hostname`). The resulting state name is stable across provision and destroy because the same order parameter value is stored on the infrastructure element and reused at decommission time.
- **Upstream dependencies:** The optional `upstreamSuffix` field lets a step declare which preceding step it depends on. The orchestrator can use this to gate execution — e.g. DNS step waits for VM step outputs (IP address).
- **Complements, not replaces, product webhooks:** `triggerProductWebhooks` and `triggerPipelineStacks` fire in parallel; their pipeline IDs are merged in `pipeline_id`. Existing single-webhook setups are unaffected. Pipeline stacks are opt-in per product+environment.
- **Security:** The `webhook_token` column is excluded from all API responses via a shared `publicColumns` projection in the service layer. The secret is written to the database on creation/update but never returned to the frontend.

Alternative considered: Storing the full `.gitlab-ci.yml` snippet in the database and injecting it into the orchestrator pipeline at trigger time. Rejected because CI YAML injection requires the orchestrator to dynamically generate pipeline definitions (a GitLab EE feature or a complex custom script), whereas passing `PIPELINE_STACK` JSON is a simple REST variable and works with any GitLab tier.

3.12 JWT Authentication

Decision: The backend issues signed JWTs (HS256, 24-hour expiry) via the `jose` library. The frontend stores the JWT in a NextAuth v5 server-side session cookie and forwards it as a `Bearer` token on every backend API call.

Rationale:

- **Stateless backend:** No session table, no Redis — every authenticated request is self-contained. The backend verifies the JWT signature and extracts the user identity and role from the token payload without a database lookup.
- **Clean separation:** The backend exposes a pure REST interface; the frontend manages session lifecycle via NextAuth v5. The JWT is the only shared contract between the two containers.
- **Horizontal scaling:** Any backend replica can verify any token — no sticky sessions or shared session store required.

- **Logout:** Because JWTs are stateless, logout is implemented by deleting the NextAuth session cookie on the frontend. The short 24-hour expiry limits the window for a captured token. A token revocation list can be added later if needed.

Secret management: `JWT_SECRET` (backend, min. 32 chars) signs and verifies tokens. `NEXTAUTH_SECRET` (frontend) encrypts the NextAuth session cookie. Both must be kept secret and rotated on breach.

3.13 Password Hashing: bcrypt

Decision: Local user passwords are hashed with bcrypt (cost factor 12) via the `bcryptjs` npm package.

Rationale:

- bcrypt is an adaptive, memory-hard function specifically designed for password storage; it is resistant to GPU-accelerated dictionary attacks.
- Cost factor 12 takes approximately 250 ms on modern hardware — slow enough to deter brute-force attacks while still acceptable for interactive login.
- `bcryptjs` is a pure-JavaScript implementation with no native bindings, making it portable across Node.js versions and compatible with Next.js standalone output.
- Passwords are never stored in plaintext or with reversible encryption.

Alternative considered: Argon2id. More modern, but bcrypt is widely understood, well-tested in Node.js, and sufficient for the low login-volume use case.

Part II

Infrastructure as Code

Chapter 4

GitLab Repository Concept

4.1 Overview

The IaC layer is a single GitLab repository, `infra-templates`. It contains reusable OpenTofu modules and deployable templates for each supported provider. The portal triggers the repository's entry pipeline with a `TEMPLATE` variable; the entry pipeline dispatches to the matching child pipeline inside the same repository.

Table 4.1: infra-templates repository layout

Path	Purpose
`.gitlab-ci.yml`	Entry pipeline — reads `TEMPLATE` and dispatches to the matching child pipeline
`.ci/base.gitlab-ci.yml`	Shared validate → plan → apply stages included by every template
`.ci/generate_stack.py`	Orchestrator script — reads `PIPELINE_STACK` JSON and generates a sequential child pipeline
`modules/`	Reusable OpenTofu modules (one per resource type)
`templates/<provider>/<name>/`	Deployable templates; each includes the base CI and sets `TEMPLATE_DIR`
`templates/orchestrator/`	Orchestrator template for pipeline stack execution

4.2 Repository File Structure

```
infra-templates/
+-- .gitlab-ci.yml                    # entry pipeline (dispatch on TEMPLATE)
+-- .ci/
|   +-- base.gitlab-ci.yml           # shared validate/plan/apply
|   +-- generate_stack.py            # pipeline stack generator
+-- modules/                         # reusable building blocks
|   +-- linode-instance/
|   +-- linode-firewall/
```

```
|   +-- linode-dns-record/
|   +-- vsphere-vm/
+-- templates/                        # deployable products
    +-- linode/
    |   +-- virtual-machine/          # instance + per-VM firewall
    |   +-- firewall/                 # standalone firewall
    |   +-- dns-record/               # Linode DNS record
    +-- vsphere/
    |   +-- virtual-machine/          # vSphere VM clone
    +-- orchestrator/                 # pipeline stack orchestrator
```

Each template directory contains the OpenTofu files (`main.tf`, `variables.tf`, `outputs.tf`, `backend.tf`) and a minimal `.gitlab-ci.yml` that includes the shared base:

```
include:
  - local: .ci/base.gitlab-ci.yml

variables:
  TEMPLATE_DIR: templates/linode/virtual-machine
```

Listing 4.1: Minimal template .gitlab-ci.yml

4.3 Entry Pipeline and Dispatch

The root `.gitlab-ci.yml` has a single `dispatch` stage. It reads the `TEMPLATE` variable and triggers the child pipeline for the matching template via `trigger: include: local:`. New templates are registered by adding one trigger job with a matching `rules` condition:

```
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "trigger"

stages:
  - dispatch

trigger-linode-virtual-machine:
  stage: dispatch
  trigger:
    include:
      - local: templates/linode/virtual-machine/.gitlab-ci.yml
    strategy: depend
  rules:
    - if: $TEMPLATE == "linode/virtual-machine"

trigger-orchestrator:
  stage: dispatch
  trigger:
    include:
      - local: templates/orchestrator/.gitlab-ci.yml
    strategy: depend
  rules:
```

```
    - if: $TEMPLATE == "orchestrator"
```

Listing 4.2: Entry pipeline — root .gitlab-ci.yml (excerpt)

4.4 Base CI Pipeline

Every template child pipeline inherits three stages from `.ci/base.gitlab-ci.yml`:

1. **validate** — `tofu validate`: checks HCL syntax and provider schema compliance. Runs without backend initialisation.
2. **plan** — promotes product parameters to `TF_VAR_*` via a shell loop, initialises the GitLab HTTP backend using `CI_PROJECT_ID` and `TF_STATE_NAME`, then runs `tofu plan -out=tfplan` (or `tofu plan -destroy` when `TF_ACTION=destroy`).
3. **apply** — same init, then `tofu apply tfplan`. The platform reads this job's log to parse `Outputs:` and store them on the infrastructure element.

Variable promotion: Non-system CI variables are automatically promoted to `TF_VAR_<lowercase>` so OpenTofu picks them up without extra configuration in `variables.tf`. Cloud credentials (`TF_VAR_linode_token`, etc.) must be set as GitLab project or group CI/CD variables — never as product parameters.

4.5 State Management

State is stored inside the `infra-templates` project itself using GitLab Managed Terraform State. The backend uses `CI_PROJECT_ID` (injected automatically by GitLab CI) as the project reference — no separate state-hosting project is needed.

State naming convention: Each infrastructure element gets a unique state name composed of the `TF_STATE_NAME` value supplied by the portal. For single-template orders this is typically the resource label (e.g. `web-01`). For pipeline stacks, `generate_stack.py` appends a per-step suffix (`web-01-vm`, `web-01-fw`, `web-01-dns`) so each step has isolated state and independent locking.

The state backend URL follows this pattern:

```
https://<gitlab-host>/api/v4/projects/<CI_PROJECT_ID>/terraform/state/<
    ↪ TF_STATE_NAME>
```

Backend configuration is injected at `tofu init` time via `-backend-config` flags in `base.gitlab-ci.yml` — no credentials are hard-coded in `backend.tf`.

4.6 Orchestrator Template (Pipeline Stacks)

`templates/orchestrator/` is the entry point for pipeline stacks (see Section **??**). Its `.gitlab-ci.yml` has two stages:

1. **generate** — `.ci/generate_stack.py` reads `PIPELINE_STACK` (JSON from the portal) and `TF_STATE_NAME`, and writes `generated-pipeline.yml` — a sequential child pipeline with one trigger job per step, each wired with the correct `TEMPLATE_DIR`, `TF_STATE_NAME`, and `TF_ACTION`. On `TF_ACTION=destroy` the step order is reversed so resources are torn down in the correct dependency order.
2. **execute** — triggers `generated-pipeline.yml` as a child pipeline with `strategy: depend` and waits for all steps to complete.

Note

The only required GitLab CI/CD variables to set are cloud provider credentials with the `TF_VAR_` prefix (e.g. `TF_VAR_linode_token`). `CI_JOB_TOKEN` and `CI_API_V4_URL` are injected automatically by GitLab CI.

4.7 Webhook Integration with the Webshop

1. An Admin approves an order in the webshop (or places one directly).
2. For each product webhook or pipeline stack configured, the webshop sends:

```
POST <gitlab-trigger-url>
X-Gitlab-Token: <secret>
Content-Type: application/json

{
  "ref": "main",
  "variables": [
    {"key": "TEMPLATE",     "value": "linode/virtual-machine"},
    {"key": "TF_STATE_NAME", "value": "web-01"},
    {"key": "TF_ACTION",    "value": "apply"},
    {"key": "ORDER_ID",     "value": "99"},
    {"key": "HOSTNAME",     "value": "web-01"},
    ...
  ]
}
```

3. GitLab starts a pipeline for the `main` branch. The entry pipeline reads `TEMPLATE` and dispatches to the matching child pipeline.
4. The child pipeline's base CI promotes product parameters to `TF_VAR_*` automatically, then runs `tofu validate`, `tofu plan`, `tofu apply`.
5. On completion (success or failure), GitLab calls the portal's pipeline webhook at `/api/webhooks/{provider}/pipeline`, which updates the order and infrastructure status immediately.

4.8 Template Dependencies

Within a pipeline stack, templates can depend on each other via the `upstreamSuffix` field in `PIPELINE_STACK`. The orchestrator passes the upstream step's state name as

`VM_STATE_NAME` to the downstream template, which can then read shared outputs via a `terraform_remote_state` data source:

```
# In a dns-record template that depends on an upstream vm step
data "terraform_remote_state" "vm" {
  backend = "http"
  config = {
    address = "${var.ci_api_url}/projects/${var.ci_project_id}/terraform/state/${
        ↪ var.vm_state_name}"
    username = "gitlab-ci-token"
    password = var.ci_job_token
  }
}

resource "linode_domain_record" "a" {
  domain_id = var.domain_id
  name      = var.hostname
  record_type = "A"
  target    = data.terraform_remote_state.vm.outputs.ip_address
}
```

`ci_api_url`, `ci_project_id`, and `ci_job_token` are exported as `TF_VAR_*` by the base CI automatically.

Chapter 5

OpenTofu Modules and Templates

The `infra-templates` repository separates reusable resource modules (`modules/`) from deployable product templates (`templates/`). Templates compose one or more modules and are the unit the portal dispatches to.

5.1 Reusable Modules

5.1.1 linode-instance

Purpose: Create a single Linode VM instance.

Key inputs: `label`, `region` (default `eu-central`), `type` (default `g6-nanode-1`), `image` (default `linode/ubuntu22.04`), `root_pass` (sensitive, from GitLab CI/CD variable), `authorized_key` (optional SSH public key), `tags`.

Outputs: `id` (numeric), `ip_address`, `ipv6`, `label`.

5.1.2 linode-firewall

Purpose: Create a Linode firewall and attach it to a VM.

Key inputs: `label`, `linode_id` (numeric ID from `linode-instance`), `inbound_ports_csv` (default `"22"`).

Design: A single variable accepts a CSV list of TCP ports so orderers can open multiple ports without defining separate firewall rule resources.

5.1.3 linode-dns-record

Purpose: Create an A record in a Linode DNS domain.

Key inputs: `domain_id`, `hostname`, `ip_address`, `ttl_sec` (default 300).

Typical use: Downstream step in a pipeline stack — reads the upstream VM's IP from `terraform_remote_state` and registers a DNS record.

5.1.4 vsphere-vm

Purpose: Clone a vSphere VM from a template.

Key inputs: `name`, `datacenter`, `cluster`, `datastore`, `template_name`, `network` (default `"VM Network"`), `num_cpus` (default 2), `memory_mb` (default 2048), `disk_size_gb` (default 40), `ipv4_address` (optional static IP), `ipv4_gateway`.

Credentials: `vsphere_server`, `vsphere_user`, `vsphere_password` must be set as GitLab group/project CI/CD variables with the `TF_VAR_` prefix.

5.2 Deployable Templates

5.2.1 linode/virtual-machine

Portal template name: `linode/virtual-machine`

Composes `linode-instance` and `linode-firewall` to provision a Linode VM with its own firewall in a single pipeline run. One OpenTofu state per instance (`TF_STATE_NAME` = the `hostname` order parameter).

Product parameters to define in the portal:

Parameter	Default	Description
`hostname`	—	Instance label / state key
`region`	`eu-central`	Linode region
`instance_type`	`g6-nanode-1`	Instance plan
`image`	`linode/ubuntu22.04`	OS image
`inbound_ports_csv`	`22`	Open TCP ports

5.2.2 vsphere/virtual-machine

Portal template name: `vsphere/virtual-machine`

Clones a VM from a vSphere template. Product parameters map to the `vsphere-vm` module variables.

5.2.3 linode/firewall and linode/dns-record

Standalone templates for environments where firewall rules or DNS records are ordered independently (rather than as part of a pipeline stack). Both include `base.gitlab-ci.yml` and set the corresponding `TEMPLATE_DIR`.

5.2.4 orchestrator

Portal template name: `orchestrator`

Entry point for pipeline stacks. Triggered with `TEMPLATE=orchestrator` by the portal's `triggerPipelineStacks()` function. Runs `.ci/generate_stack.py` to convert `PIPELINE_STACK` JSON into a sequential GitLab child pipeline, then executes it. See Section 4.7 for how the generated pipeline resolves inter-step state dependencies.

Part III

Administrator Guide

Chapter 6

Initial Setup & Installation

6.1 Prerequisites

- Docker 24+ (or a Kubernetes cluster)
- PostgreSQL 15+ (external or containerized)
- SMTP server credentials (optional, can be configured later)
- Microsoft Entra ID application registration for OIDC (optional for local-only operation)
- Linode API token with Read/Write scope
- GitLab instance with webhook support

6.2 Environment Variables

The system is configured via two `.env` files — one per application container. Copy the provided example files and populate the values before starting:

```
cp apps/backend/.env.example apps/backend/.env
cp apps/frontend/.env.example apps/frontend/.env
```

Backend (`apps/backend/.env`):

Table 6.1: Backend environment variables

Variable	Description	Required
DATABASE_URL	PostgreSQL DSN	Yes
JWT_SECRET	HS256 signing secret (min. 32 chars)	Yes
ADMIN_EMAIL	Initial Root account email	Yes
ADMIN_PASSWORD	Initial Root account password	Yes
FRONTEND_URL	Frontend origin for CORS (default: `http://localhost:3000`)	No
SMTP_HOST	SMTP server hostname	No
SMTP_PORT	SMTP port (default: `1025`)	No
SMTP_FROM	Sender email address	No
SMTP_USER	SMTP authentication username	No
SMTP_PASS	SMTP authentication password	No
SMTP_TLS	`true` to enable TLS	No
EXCHANGE_RATE_API_URL	Exchange rate API endpoint	No
ENTRA_TENANT_ID	Microsoft Entra ID tenant ID (SSO)	No
ENTRA_CLIENT_ID	Entra ID application client ID	No
ENTRA_CLIENT_SECRET	Entra ID client secret	No

Frontend (apps/frontend/.env):

Table 6.2: Frontend environment variables

Variable	Description	Required
NEXT_PUBLIC_API_URL	Backend URL reachable from the browser	Yes
API_URL	Backend URL reachable from the SSR server	Yes
NEXTAUTH_URL	Canonical public URL of the frontend	Yes
NEXTAUTH_SECRET	NextAuth session encryption secret (min. 32 chars)	Yes

Note

SMTP can be configured (and overridden) through the Admin UI. Values saved in the database take precedence over environment variables at runtime.

6.3 Deployment on a Docker Host

This section describes a production-grade deployment on a single Linux host using Docker and an Nginx reverse proxy with TLS termination.

Architecture

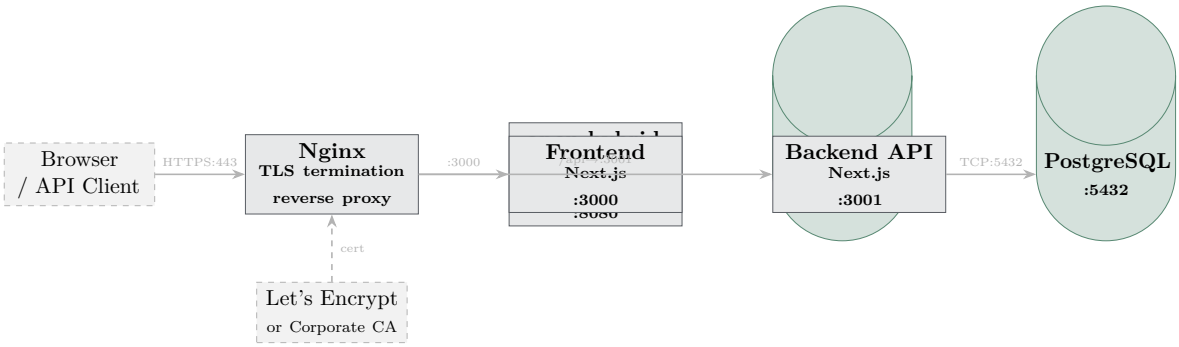

Figure 6.1: Docker Host deployment architecture

Nginx routes `/api/*` and `/api/webhooks/*` to the backend container (port 3001) and all other paths to the frontend (port 3000).

Step-by-Step Deployment Procedure

1. **Install Docker and Docker Compose.**

```
# Debian/Ubuntu
apt-get install -y docker.io docker-compose-v2
systemctl enable --now docker
```

2. **Clone the repository.**

```
git clone https://github.com/Maximilian-Maag/open-hybrid-cloud.git
cd open-hybrid-cloud
```

3. **Configure environment files.**

```
# Edit both files with production values
cp apps/backend/.env.example apps/backend/.env
cp apps/frontend/.env.example apps/frontend/.env
# Required backend vars: DATABASE_URL, JWT_SECRET, ADMIN_EMAIL,
    ↪ ADMIN_PASSWORD
# Required frontend vars: NEXT_PUBLIC_API_URL, API_URL, NEXTAUTH_URL,
    ↪ NEXTAUTH_SECRET
```

4. **Start all containers.**

```
docker compose -f infra/docker-compose.yml up -d
```

   The schema is applied automatically on first startup (Drizzle push runs as part of the backend bootstrap). The root user is created from `ADMIN_EMAIL` / `ADMIN_PASSWORD` on the first `GET /api/health` call.

5. **Obtain a TLS certificate.**

   Option A — Let's Encrypt (public domain):

```
apt-get install -y certbot python3-certbot-nginx
certbot --nginx -d shop.example.com
```

Option B — Corporate CA (internal PKI): Place the signed certificate and key at the paths referenced in `infra/docker-host/nginx.conf` (see below) and restart Nginx.

6. **Configure Nginx as TLS-terminating reverse proxy.**

   Listing 6.1 shows a minimal but production-ready Nginx configuration:

```
server {
    listen 80;
    server_name shop.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name shop.example.com;

    ssl_certificate   /etc/letsencrypt/live/shop.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/shop.example.com/privkey.pem;
    ssl_protocols     TLSv1.2 TLSv1.3;
    ssl_ciphers       HIGH:!aNULL:!MD5;

    client_max_body_size 10m;

    # Backend API (REST + webhooks)
    location /api/ {
        proxy_pass       http://ohc-backend:3001;
        proxy_set_header Host            $host;
        proxy_set_header X-Real-IP       $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    # Frontend (Next.js UI)
    location / {
        proxy_pass       http://ohc-frontend:3000;
        proxy_set_header Host            $host;
        proxy_set_header X-Real-IP       $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }
}
```

Listing 6.1: infra/nginx/default.conf

7. **Reload Nginx.**

```
nginx -t && systemctl reload nginx
```

8. **Verify the deployment.**

   Navigate to https://shop.example.com and log in with ADMIN_EMAIL / ADMIN_PASSWORD. Change the password immediately under **Settings → My Profile**.

9. **(Optional) Enable automatic certificate renewal.**

```
# Certbot installs a systemd timer automatically
systemctl status certbot.timer
```

Backup & Restore

```
# Backup PostgreSQL
docker exec ohc-postgres pg_dump -U postgres open_hybrid_cloud \
  | gzip > backup-$(date +%Y%m%d).sql.gz

# Restore
zcat backup-20260101.sql.gz \
  | docker exec -i ohc-postgres psql -U postgres open_hybrid_cloud
```

6.4 Deployment on Kubernetes

This section covers deploying the webshop to an existing Kubernetes cluster.

Architecture

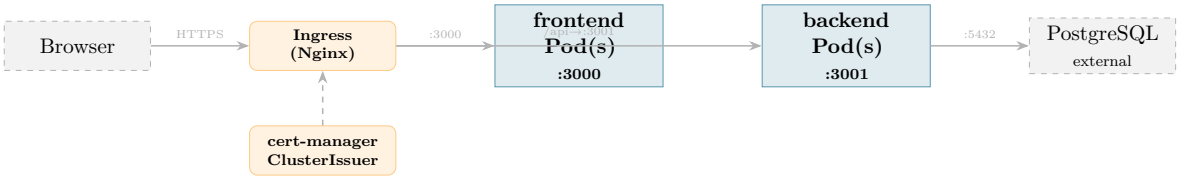

Figure 6.2: Kubernetes deployment architecture

Prerequisites

- `kubectl` and `helm` (v3.12+) configured for the target cluster
- Nginx Ingress Controller installed (`ingress-nginx`)
- cert-manager installed with a `ClusterIssuer` for Let's Encrypt or a corporate CA
- The images `maximilianmaag/open-hybrid-cloud-backend` and `maximilianmaag/open-hybri` are publicly available on Docker Hub — no `imagePullSecret` required

Deployment with Helm

The repository ships a Helm chart at `infra/helm/open-hybrid-cloud/`. It covers the full deployment in a single command and handles database migrations automatically as a pre-install/pre-upgrade hook.

```
helm upgrade --install open-hybrid-cloud \
  infra/helm/open-hybrid-cloud \
  --namespace open-hybrid-cloud \
  --create-namespace \
  --set backend.env.databaseUrl="postgres://user:pass@pg.example.com:5432/
      ↪ open_hybrid_cloud?sslmode=require" \
  --set backend.env.jwtSecret="<min-32-char-secret>" \
  --set backend.env.adminPassword="<initial-password>" \
  --set frontend.env.nextauthSecret="<min-32-char-secret>" \
  --set frontend.env.apiUrl="http://open-hybrid-cloud-backend:3001" \
  --set ingress.hosts[0].host=shop.example.com \
  --set ingress.tls[0].hosts[0]=shop.example.com \
  --set ingress.tls[0].secretName=open-hybrid-cloud-tls
```

Listing 6.2: Helm install with required values

Note

Sensitive values should be passed via a separate `secrets.values.yaml` file (not committed to version control) rather than as `-set` arguments, so they do not appear in shell history.

```
backend:
  env:
    databaseUrl: "postgres://user:pass@pg.example.com:5432/open_hybrid_cloud?
        ↪ sslmode=require"
    jwtSecret: "aabbccddeeff00112233445566778899aabbccddeeff00112233445566778899"
    adminPassword: "change-me-immediately"
frontend:
  env:
    nextauthSecret: "00112233445566778899
        ↪ aabbccddeeff00112233445566778899aabbccddeeff"
    nextauthUrl: "https://shop.example.com"
```

Listing 6.3: Example secrets.values.yaml (do not commit)

```
helm upgrade --install open-hybrid-cloud infra/helm/open-hybrid-cloud \
  --namespace open-hybrid-cloud --create-namespace \
  -f values.production.yaml \
  -f secrets.values.yaml
```

Table 6.3 lists the most important configurable values.

Table 6.3: Key Helm values

Key	Description	Default
`backend.image.repository`	Backend Docker Hub image	`maximilianmaag/open-hybrid-cloud-backend`
`backend.image.tag`	Image tag (empty = appVersion)	`""`
`backend.replicaCount`	Backend pod replicas	`2`
`backend.env.databaseUrl`	PostgreSQL DSN (required)	—
`backend.env.jwtSecret`	JWT signing secret (required)	—
`backend.env.adminPassword`	Initial root password (required)	—
`frontend.image.repository`	Frontend Docker Hub image	`maximilianmaag/open-hybrid-cloud-frontend`
`frontend.image.tag`	Image tag (empty = appVersion)	`""`
`frontend.replicaCount`	Frontend pod replicas	`2`

Step-by-Step Manual Deployment (without Helm)

1. **Create the namespace.**

```
kubectl create namespace open-hybrid-cloud
```

2. **(Optional) Create an image pull secret.**

   The container image `maximilianmaag/open-hybrid-cloud` is publicly available on Docker Hub and requires no authentication. Skip this step for standard deployments. Only required if you mirror the image to a private registry:

```
kubectl create secret docker-registry my-registry-secret \
  --namespace open-hybrid-cloud \
  --docker-server=<your-registry> \
  --docker-username=<username> \
  --docker-password=<password>
```

3. **Create the application Secret.**

```
apiVersion: v1
kind: Secret
metadata:
  name: ohc-backend-env
  namespace: open-hybrid-cloud
type: Opaque
stringData:
  DATABASE_URL: "postgres://user:password@pg.example.com:5432/
      ↪ open_hybrid_cloud?sslmode=require"
  JWT_SECRET: "000102030405060708090
      ↪ a0b0c0d0e0f101112131415161718191a1b1c1d1e1f"
  ADMIN_EMAIL: "root@example.com"
  ADMIN_PASSWORD: "changeme"
---
apiVersion: v1
kind: Secret
metadata:
  name: ohc-frontend-env
  namespace: open-hybrid-cloud
type: Opaque
stringData:
  NEXTAUTH_SECRET: "
      ↪ f0e1d2c3b4a5968700010203040506 07f0e1d2c3b4a596870001020304050607"
  NEXTAUTH_URL: "https://shop.example.com"
  NEXT_PUBLIC_API_URL: "https://shop.example.com/api"
  API_URL: "http://ohc-backend:3001"
```

Listing 6.4: Kubernetes Secrets for backend and frontend

4. **Apply the database schema.**

   The schema is pushed automatically by Drizzle on backend startup. No separate migration job is required. Verify via the health endpoint once the backend pod is running:

```
kubectl exec -n open-hybrid-cloud deploy/ohc-backend -- \
  wget -qO- http://localhost:3001/api/health
# Expected: {"status":"ok"}
```

5. **Deploy the application.**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: open-hybrid-cloud
  namespace: open-hybrid-cloud
spec:
  replicas: 2
  selector:
    matchLabels:
      app: open-hybrid-cloud
  template:
    metadata:
      labels:
        app: open-hybrid-cloud
    spec:
      containers:
        - name: app
          image: maximilianmaag/open-hybrid-cloud:latest
          ports:
            - containerPort: 8080
          envFrom:
            - secretRef:
                name: open-hybrid-cloud-env
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 10
            periodSeconds: 30
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
---
apiVersion: v1
kind: Service
metadata:
  name: open-hybrid-cloud
  namespace: open-hybrid-cloud
spec:
```

```
  selector:
    app: open-hybrid-cloud
  ports:
    - port: 80
      targetPort: 8080
```

Listing 6.5: Deployment and Service

6. **Configure the Ingress with TLS.**

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: open-hybrid-cloud
  namespace: open-hybrid-cloud
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - shop.example.com
      secretName: open-hybrid-cloud-tls
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: open-hybrid-cloud
                port:
                  number: 80
```

Listing 6.6: Ingress with cert-manager TLS

7. **(Optional) Configure Horizontal Pod Autoscaler.**

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: open-hybrid-cloud
  namespace: open-hybrid-cloud
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: open-hybrid-cloud
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
```

```
    target:
      type: Utilization
      averageUtilization: 70
```

Listing 6.7: HorizontalPodAutoscaler

8. **Apply all manifests and verify.**

```
kubectl apply -f secret.yaml
kubectl apply -f deployment.yaml
kubectl apply -f ingress.yaml
kubectl apply -f hpa.yaml   # optional

kubectl rollout status deployment/open-hybrid-cloud -n open-hybrid-cloud
kubectl get ingress -n open-hybrid-cloud
```

Note

The backend is stateless — all state lives in PostgreSQL. Multiple replicas can run concurrently without coordination; CI webhook events are idempotent so duplicate deliveries are handled safely.

6.5 Database Schema Migrations

The schema is managed by Drizzle ORM. Apply it before first start or after any schema change:

```
make db-push
```

This runs `drizzle-kit push` which compares the TypeScript schema definition against the live database and applies the diff.

6.6 First Login

1. Navigate to the webshop URL.
2. Click **Local Login** (not **Sign in with Microsoft**).
3. Enter the email and password from `ADMIN_EMAIL` and `ADMIN_PASSWORD`.
4. Navigate to **Settings** → **My Profile** and change the default password immediately.

Chapter 7

System Configuration

7.1 GitLab Sources

Under **Administration → GitLab Sources**:

1. Click **Add New Source**.
2. Enter a **Name** (label), the instance **URL**, and a **Personal Access Token** with `read_api` scope.
3. Save. The source is now available when creating environments and when browsing repositories for parameter import.

7.2 Deployment Environments

Under **Administration → Deployment Environments**:

1. Click **Add New Environment**.
2. Enter a **Name** (e.g. *Linode Frankfurt Production*), select the **GitLab Source**, and enter the **Webhook URL** and **Webhook Token**.
3. Save. The environment becomes available when assigning products.

Note

The Webhook URL points to a specific GitLab project pipeline trigger endpoint, e.g.: `https://gitlab.example.com/api/v4/projects/42/ref/main/trigger/pipeline`

7.3 SMTP Configuration

Under **Administration → Email** (`/admin/smtp`):

Enter host, port, sender address, username, password, and TLS mode. Click **Save**. The notification service reloads credentials immediately without a server restart.

7.4 AI Translation

Under **Administration → AI Translation** (`/admin/ai-config`):

1. Select a **Provider**: Claude, OpenAI, Azure OpenAI, Ollama, or LocalAI.
2. Enter the **API endpoint** and **API key**. For Ollama/LocalAI, the endpoint is a local URL and no key is required.
3. Enter the **model name** (e.g. `claude-opus-4-7`, `gpt-4o`, `llama3.1`).
4. Click **Save**. The translator instance is replaced immediately — the next translation request uses the new provider.

7.5 Branding

Under **Administration → Shop Design** (`/admin/branding`):

- **Shop Name** and **Subtitle** — shown in the header.
- **Primary Color** — header, footer, navigation (default: `#0f172a`).
- **Accent Color** — buttons and call-to-action elements (default: `#0ea5e9`).
- **Logo** — PNG or SVG, displayed in the header; replaces the shop name text if present.
- **Imprint** — HTML-formatted legal imprint text, shown at `/impressum`.

All branding values are stored in a singleton database row and cached in-memory with a mutex for read performance.

7.6 Currencies

Under **Administration → Currencies**:

1. The **base currency** (default EUR) is the currency in which all product prices are stored.
2. Optionally configure an exchange rate API key and endpoint.
3. Click **Update Rates** to fetch live rates.
4. Individual rates can be overridden manually for fixed-rate environments.

Prices are displayed in the user's locale currency by looking up the browser locale and applying the stored rate.

Chapter 8

User Management

8.1 Roles

Table 8.1: User roles and capabilities

Role	Description	Auth method	DB role
Admin	Full access; direct ordering; approval	SSO	`admin`
Root	Catalog & config; no ordering	Local	`root`
Project Manager	Orders own infra; approval required	SSO	`project_leader`

8.2 Creating a Local Account

Under **Administration → Users**:

1. Click **New User**.
2. Enter **Name**, **Email**, **Password**, and select a **Role**.
3. Click **Save**.

SSO users are created automatically on first login.

8.3 Editing Users

Click **Edit** next to a user to change name, email, or role. The password can only be changed by the user themselves under **Settings → My Profile**.

8.4 Deactivating Users

Click **Deactivate** on the user edit page. Deactivated users cannot log in. Their orders and projects remain visible in the system.

Chapter 9

Audit Log

The audit log under `/audit` is an append-only, tamper-evident record of all system actions.

9.1 Logged Events

Table 9.1: Audited actions

Action	Trigger
`order_placed`	User submits an order
`order_approved`	Admin approves
`order_rejected`	Admin rejects (with comment)
`order_deployed`	Pipeline completes successfully
`order_failed`	Pipeline fails
`decommission_started`	User triggers decommission
`decommission_complete`	Destroy pipeline succeeds
`user_login`	Successful authentication
`user_created`	Admin creates a user
`config_changed`	System configuration updated

9.2 Filtering

Use the filter form to narrow results by:

- **Date range** (from / to)
- **User**
- **Action type**

9.3 Export

Click **Export CSV** or **Export PDF** to download the currently filtered audit log for compliance reporting.

Part IV

Developer Guide

Chapter 10

Development Environment

10.1 Prerequisites

- Node.js 22+
- pnpm 9+ (`npm install -g pnpm`)
- Docker & Docker Compose
- `pdflatex` or `lualatex` (for handbook compilation, optional)

10.2 Quick Start

```
# Clone and enter the repo
git clone https://github.com/Maximilian-Maag/open-hybrid-cloud.git
cd open-hybrid-cloud

# Install all workspace dependencies
make install

# Copy environment files (pre-filled for local dev)
cp apps/backend/.env.example apps/backend/.env
cp apps/frontend/.env.example apps/frontend/.env

# Start infrastructure (postgres, mailpit, wiremock, structurizr)
make dev

# Apply database schema
make db-push

# Start frontend + backend dev servers
pnpm dev
```

The frontend is available at http://localhost:3000, the backend API at http://localhost:3001, and the API documentation at http://localhost:3001/api/docs.

10.3 Makefile Targets

Table 10.1: Makefile targets

Target	Action
`make install`	Install all workspace dependencies
`make dev`	Start infra containers (postgres, mailpit, wiremock, structurizr)
`make dev-down`	Stop infra containers
`make build`	Build all apps
`make lint`	Lint all apps
`make type-check`	TypeScript type-check all apps
`make docker-build`	Build both Docker images locally
`make db-push`	Push Drizzle schema to the database
`make db-studio`	Open Drizzle Studio
`make docs`	Compile the LaTeX handbook to PDF
`make clean`	Remove build artefacts

Chapter 11

Codebase Structure

11.1 Directory Layout

```
open-hybrid-cloud/
+-- apps/
|   +-- backend/               # Next.js API-only app (port 3001)
|   |   +-- src/
|   |   |   +-- app/api/       # Thin route handlers
|   |   |   +-- lib/
|   |   |       +-- auth/      # JWT sign/verify, role middleware
|   |   |       +-- bootstrap/  # Root user seed on first start
|   |   |       +-- ci/        # CI provider clients + webhook triggering
|   |   |       +-- db/        # Drizzle client, schema, query helpers
|   |   |       +-- http.ts    # toResponse() -- maps Result<T> to NextResponse
|   |   |       +-- notification/ # Nodemailer email notifications
|   |   |       +-- services/  # Domain services: all business logic
|   |   |       |   +-- admin/  # Admin-domain services (catalog, config, ...)
|   |   |       +-- webhook/   # CI pipeline event handler
|   |   +-- Dockerfile
|   |   +-- drizzle.config.ts
|   +-- frontend/              # Next.js UI app (port 3000)
|       +-- src/
|       |   +-- app/           # App Router pages
|       |   +-- lib/
|       |       +-- api.ts     # Typed fetch wrappers
|       |       +-- auth.ts    # NextAuth.js config
|       +-- Dockerfile
+-- packages/
|   +-- types/                 # Shared TypeScript interfaces
+-- infra/
|   +-- docker-compose.dev.yml # Local dev: postgres, mailpit, wiremock,
    ↪ structurizr
|   +-- docker-compose.yml     # Docker host deployment
|   +-- nginx/                 # Reverse proxy config
|   +-- wiremock/              # External API stubs for local dev
|   +-- helm/                  # Kubernetes Helm chart
+-- e2e/                       # Playwright end-to-end tests
+-- docs/
|   +-- architecture/
```

```
|   |   +-- workspace.dsl       # Structurizr C4 architecture
|   +-- handbook.tex
|   +-- handbook.pdf            # compiled output
+-- Makefile
+-- package.json
+-- pnpm-workspace.yaml
```

11.2 Layered Architecture

The backend follows a strict layered architecture with downward-only dependencies:

Route Handler → Service → Drizzle ORM → PostgreSQL

Route handlers (src/app/api/) Thin HTTP shells: authenticate request, parse and validate body with Zod, call the appropriate service function, return `toResponse(result)`. No business logic. No SQL.

Service layer (src/lib/services/) All business rules and orchestration. Returns `Result<T>` — a discriminated union of `Ok<T>` or `Err`. Triggers notifications, audit writes, and CI webhook dispatches.

Shared query helpers (src/lib/db/queries.ts) Reusable read-only DB helpers (`findProductName`, `findUserEmail`, `findAdminEmails`, `findCiSourceForEnv`) shared across services and the webhook handler.

Drizzle ORM (src/lib/db/) Type-safe query builder backed by `drizzle-kit push` for schema management. The schema definition is the single source of truth; no separate migration files are maintained.

Result<T> and toResponse

All service functions return a `Result<T>` discriminated union defined in src/lib/services/result.ts:

```
export type Ok<T> = { ok: true; data: T }
export type Err  = { ok: false; status: number; message: string }
export type Result<T> = Ok<T> | Err

export const ok = <T>(data: T): Ok<T> => ({ ok: true, data })
export const err = (status: number, message: string): Err =>
  ({ ok: false, status, message })
```

The helper `toResponse(result, successStatus?)` in src/lib/http.ts maps a `Result` to a `NextResponse`, keeping route handlers free of branching logic:

```
export async function GET(req: NextRequest) {
  const session = await requireAuth(req)
  if (!isAuth(session)) return session
  return toResponse(await listOrders(session))
}
```

11.3 Database Schema Management

The schema is defined in `apps/backend/src/lib/db/schema.ts` using Drizzle ORM's TypeScript schema builder. Drizzle Kit's *push* mode applies the schema directly to the database — no separate SQL migration files are maintained.

```
make db-push      # applies schema changes to the connected database
make db-studio    # opens Drizzle Studio (visual schema browser)
```

Important

In production, prefer `drizzle-kit migrate` with generated migration files to retain a full change history. `push` is suitable for development and initial deployments.

Chapter 12

Testing

The project uses two complementary test layers, both run automatically in CI via `pnpm test` and `pnpm exec playwright test`.

12.1 Service and Integration Tests (Vitest)

Tests live alongside the code they cover as `*.test.ts` files and run under Vitest. They target service functions directly — no HTTP layer, no mocks for the database.

Test setup

A global setup file (`src/test/setup.ts`) starts a real PostgreSQL container once per test run (via testcontainers) and exposes the connection URL in `process.env.DATABASE_URL`. A global `beforeEach` truncates all tables and resets sequences before every test so each case starts with a clean slate.

```
// vitest.config.ts
export default defineConfig({
  test: {
    setupFiles: ['./src/test/setup.ts'],
    globalSetup: './src/test/globalSetup.ts',
  },
})
```

Session helpers

Tests construct a `SessionUser` directly with a small helper instead of going through the auth layer:

```
const makeSession = (overrides?: Partial<SessionUser>): SessionUser => ({
  id: 1, email: 'test@example.com', name: 'Test',
  role: 'admin', ...overrides,
})
```

Mocking side effects

Services that trigger CI pipelines or send emails are tested with `vi.mock()` to isolate the unit under test:

```
vi.mock('@/lib/ci/webhooks', () => ({
  triggerProductWebhooks: vi.fn().mockResolvedValue(['pipe-1']),
}))
vi.mock('@/lib/notification', () => ({
  sendOrderCreated: vi.fn(),
  sendApprovalRequest: vi.fn(),
}))
```

Run the test suite:

```
pnpm --filter backend test      # watch mode
pnpm --filter backend test --run # single run (CI)
```

12.2 End-to-End Tests with Playwright

End-to-end tests live in `e2e/` and drive a real **Chromium** browser via Playwright for TypeScript.

Table 12.1: E2E test files

File	What is verified
`login.spec.ts`	Login success, invalid credentials, logout
`catalog.spec.ts`	Page loads, empty state, product navigation
`admin.spec.ts`	Admin-only catalog and configuration pages

Running the tests

```
# Requires running frontend + backend + postgres
pnpm dev          # in one terminal
npx playwright test # in another terminal

# Headed mode -- watch the browser
npx playwright test --headed
```

Adding a new E2E test

1. Create a new `*.spec.ts` file in `e2e/`.
2. Use `page.goto()` to navigate; authenticate via the login form if needed.
3. Prefer stable selectors: `data-testid`, ARIA roles, or exact button text. Avoid CSS classes (they change).
4. After a form submit, await `page.waitForURL()` or `expect(page).toHaveURL()` before asserting page state.

Chapter 13

Database Schema

The schema is managed by Drizzle ORM. The authoritative definition is in `apps/backend/src/lib/db/schema.ts`; `make db-push` (`drizzle-kit push`) applies any schema changes to the database. The table below shows the current effective schema.

13.1 Table Overview

Table 13.1: All database tables

Table	Purpose
`users`	User accounts (local + SSO) with role-based access
`categories`	Product catalogue categories
`products`	Infrastructure products offered in the catalogue
`product_translations`	Multi-language name + description per product
`parameters`	Configurable inputs for global/category/product scope
`ci_sources`	CI provider credentials (GitLab / GitHub / Bitbucket)
`deployment_environments`	Named environments linked to a CI source
`product_environments`	Junction: which products are available in which environment (with price)
`product_webhooks`	Optional per-product pipeline trigger overrides
`cost_centers`	Billing cost centres referenced by projects and orders
`projects`	User-owned projects that group orders
`orders`	Individual infrastructure provisioning requests
`infrastructure_elements`	Deployed instances tracked per order
`audit_log`	Append-only log of significant user actions
`exchange_rates`	Cached currency conversion rates
`branding`	Single-row branding config (logo, colours, texts)
`app_config`	Single-row application config (SMTP, AI)

13.2 Detailed Schema

Each table is described with its columns (post all migrations), primary key, foreign keys, and notable constraints.

`users`

Column	Type	Default	Notes
`id`	BIGSERIAL	—	PK
`email`	TEXT	—	NOT NULL, UNIQUE
`name`	TEXT	—	NOT NULL
`role`	TEXT	—	CHECK: `admin`, `project_manager`, `root`
`sso_sub`	TEXT	NULL	UNIQUE; set for SSO accounts
`password_hash`	TEXT	NULL	set for local accounts
`active`	BOOLEAN	TRUE	soft-disable without deletion
`created_at`	TIMESTAMPTZ	NOW()	

Constraint: `sso_sub IS NOT NULL OR password_hash IS NOT NULL` (every account must have at least one credential).

categories

Column	Type	Default	Notes
`id`	BIGSERIAL	—	PK
`name`	TEXT	—	NOT NULL
`display_order`	INT	0	controls sidebar sort order

products

Column	Type	Default	Notes
`id`	BIGSERIAL	—	PK
`category_id`	BIGINT	—	NOT NULL, FK → `categories(id)` ON DELETE CASCA
`base_language`	TEXT	`'de'`	fallback language for translations
`image`	BYTEA	NULL	product image stored inline
`created_at`	TIMESTAMPTZ	NOW()	

product_translations

Column	Type	Default	Notes
`product_id`	BIGINT	—	PK, FK → `products(id)` ON DELETE CASCADE
`language_code`	TEXT	—	PK, e.g. `de`, `en`, `fr`
`name`	TEXT	—	NOT NULL
`description`	TEXT	`''`	NOT NULL

parameters

Column	Type	Default	Notes
`id`	BIGSERIAL	—	PK
`scope`	TEXT	—	CHECK: `global`, `category`, `product`
`scope_id`	BIGINT	0	ID of the scoped entity (0 for global)
`environment_id`	BIGINT	0	0 = applies to all environments
`name`	TEXT	—	NOT NULL; variable name passed to pipeline
`type`	TEXT	—	CHECK: `string`, `number`, `bool`, `dropdown`
`description`	TEXT	`''`	
`default_value`	TEXT	`''`	
`required`	BOOLEAN	FALSE	
`sensitive`	BOOLEAN	FALSE	renders as `type="password"` in UI

ci_sources

Formerly `gitlab_sources`. Supports multiple CI providers via the `provider` discriminator column.

Column	Type	Default	Notes
`id`	BIGSERIAL	—	PK
`name`	TEXT	—	NOT NULL; human-readable label
`provider`	TEXT	`'gitlab'`	CHECK: `gitlab`, `github`, `bitbucket`
`url`	TEXT	—	NOT NULL; CI provider base URL
`access_token`	TEXT	—	NOT NULL

deployment_environments

Column	Type	Default	Notes
`id`	BIGSERIAL	—	PK
`name`	TEXT	—	NOT NULL
`description`	TEXT	`''`	
`ci_source_id`	BIGINT	—	NOT NULL, FK → `ci_sources(id)`
`webhook_url`	TEXT	—	NOT NULL; pipeline trigger URL
`webhook_token`	TEXT	—	NOT NULL; pipeline trigger token

product_environments

Junction table linking products to environments with pricing.

Column	Type	Default	Notes
`product_id`	BIGINT	—	PK, FK → `products(id)` CASCADE
`environment_id`	BIGINT	—	PK, FK → `deployment_environments(id)`
`price`	NUMERIC(12,2)	0	monthly price
`currency`	TEXT	`'EUR'`	ISO 4217 code
`cost_center_mode`	TEXT	`'project'`	CHECK: `project`, `select`, `overhead`
`forced_cost_center`	BOOLEAN	FALSE	locks cost centre selection in order form

product_webhooks

Optional per-product pipeline overrides; if rows exist for a product+environment pair they replace the environment's default `webhook_url`. Multiple rows are fired in ascending `exec_order`.

Column	Type	Default	Notes
id	BIGSERIAL	—	PK
product_id	BIGINT	—	NOT NULL, FK → products(id) CASCADE
environment_id	BIGINT	—	NOT NULL, FK → deployment_environments(id)
name	TEXT	—	NOT NULL; descriptive label
webhook_url	TEXT	—	NOT NULL
webhook_token	TEXT	—	NOT NULL
exec_order	INT	0	ascending; ties fire concurrently

cost_centers

Column	Type	Default	Notes
id	BIGSERIAL	—	PK
code	TEXT	—	NOT NULL, UNIQUE; billing code
name	TEXT	—	NOT NULL
active	BOOLEAN	TRUE	inactive centres are hidden in UI

projects

Column	Type	Default	Notes
id	BIGSERIAL	—	PK
name	TEXT	—	NOT NULL
description	TEXT	''	
owner_id	BIGINT	—	NOT NULL, FK → users(id)
cost_center_id	BIGINT	NULL	FK → cost_centers(id)
created_at	TIMESTAMPTZ	NOW()	

orders

Column	Type	Default	Notes
id	BIGSERIAL	—	PK
project_id	BIGINT	—	NOT NULL, FK → projects(id) CASCADE
product_id	BIGINT	—	NOT NULL, FK → products(id) CASCADE
environment_id	BIGINT	—	NOT NULL, FK → deployment_environment
user_id	BIGINT	—	NOT NULL, FK → users(id)
status	TEXT	'pending_approval'	see lifecycle below
parameters	JSONB	'{}'	key/value map; passed to pipeline
cost_center_id	BIGINT	NULL	FK → cost_centers(id)
rejection_note	TEXT	''	filled on rejection
pipeline_id	JSONB	'[]'	array of GitLab pipeline IDs
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

Status lifecycle: `pending_approval` → `approved` / `rejected` → `provisioning` → `completed` / `failed` → `decommissioning` → `decommissioned`.

infrastructure_elements

One row per deployed pipeline; created when an order is approved.

Column	Type	Default	Notes
`id`	BIGSERIAL	—	PK
`order_id`	BIGINT	—	NOT NULL, FK → `orders(id)`
`project_id`	BIGINT	—	NOT NULL, FK → `projects(id)` CASCADE
`environment_id`	BIGINT	—	NOT NULL, FK → `deployment_environments`
`product_id`	BIGINT	—	NOT NULL, FK → `products(id)` CASCADE
`status`	TEXT	`'provisioning'`	mirrors order status values
`parameters`	JSONB	`'{}'`	copy of order parameters at deploy time
`pipeline_id`	JSONB	`'[]'`	tracked pipeline IDs
`outputs`	JSONB	`'{}'`	key/value map of OpenTofu outputs
`deployed_at`	TIMESTAMPTZ	NOW()	

audit_log

Append-only; rows are never updated or deleted.

Column	Type	Default	Notes
`id`	BIGSERIAL	—	PK
`user_id`	BIGINT	—	NOT NULL, FK → `users(id)`
`action`	TEXT	—	NOT NULL; e.g. `order.created`
`entity_id`	BIGINT	—	NOT NULL; PK of the referenced entity
`details`	TEXT	`''`	extra context (e.g. rejection note)
`created_at`	TIMESTAMPTZ	NOW()	

exchange_rates

Column	Type	Default	Notes
`currency_code`	TEXT	—	PK; ISO 4217, e.g. `EUR`
`rate`	NUMERIC(18,6)	—	NOT NULL; relative to base currency (EUR = 1.0)
`updated_at`	TIMESTAMPTZ	NOW()	refreshed via `POST /admin/currencies/refresh`

branding

Singleton row (`id = 1`). Written via `POST /admin/branding`.

Column	Type	Default	Notes
id	INT	1	PK, CHECK (id = 1)
logo_data	BYTEA	NULL	uploaded logo image
logo_mime	TEXT	'image/png'	MIME type of the logo
primary_color	TEXT	'#0f172a'	var(-bp) CSS token
secondary_color	TEXT	'#0ea5e9'	var(-bs) CSS token
shop_name	TEXT	''	overrides APP_NAME env var
shop_subtitle	TEXT	''	overrides APP_SUBTITLE env var
imprint_text	TEXT	''	rendered on /impressum

app_config

Singleton row (id = 1). Written via POST /admin/smtp and POST /admin/ai-config. DB values override environment variables at runtime.

Column	Type	Default	Notes
id	INT	1	PK, CHECK (id = 1)
smtp_host	TEXT	''	SMTP server hostname
smtp_port	TEXT	'587'	
smtp_from	TEXT	''	sender address
smtp_username	TEXT	''	
smtp_password	TEXT	''	stored in plain text
smtp_tls	BOOLEAN	FALSE	enable STARTTLS
ai_provider	TEXT	''	e.g. claude, openai
ai_endpoint	TEXT	''	custom API base URL
ai_api_key	TEXT	''	stored in plain text
ai_model	TEXT	''	model ID string

13.3 Foreign Key Relationships

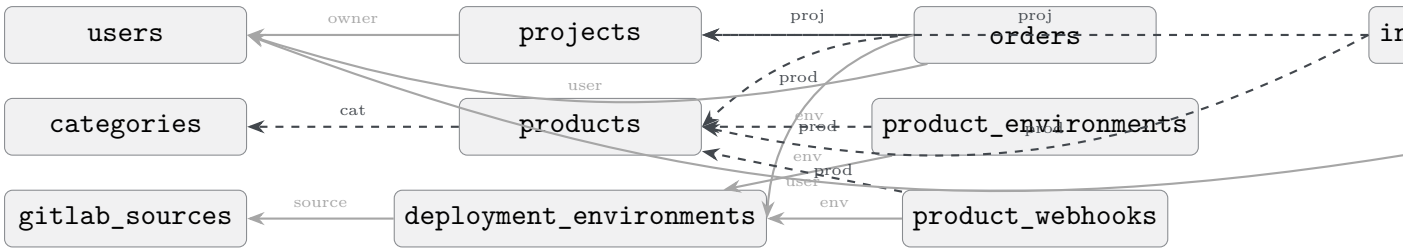

Figure 13.1: Simplified FK diagram. Dashed arrows = ON DELETE CASCADE; solid arrows = restricted.

Note

The cascade policy ensures that deleting a **product**, **project**, or **category** automatically removes all dependent rows (orders, infra elements, translations, etc.) without requiring explicit cleanup in application code.

13.4 Indexes

Table 13.2: Non-primary-key indexes

Index name	Table	Columns
idx_products_category	products	category_id
idx_orders_user	orders	user_id
idx_orders_status	orders	status
idx_orders_project	orders	project_id
idx_infra_project	infrastructure_elements	project_id
idx_infra_status	infrastructure_elements	status
idx_projects_owner	projects	owner_id
idx_audit_user	audit_log	user_id
idx_audit_action	audit_log	action
idx_parameters_scope	parameters	scope, scope_id, environment_id
idx_product_webhooks	product_webhooks	product_id, environment_id, exec_or

Chapter 14

Adding a New Feature

This chapter describes the canonical pattern for adding a feature end-to-end in the Next.js monorepo.

14.1 Example: Adding a New Admin Page

Goal: Add an admin endpoint that lists and creates widgets stored in a new `widgets` database table, and a corresponding admin UI page.

14.1.1 Step 1: Add the Table to the Drizzle Schema

In apps/backend/src/lib/db/schema.ts, add the table definition:

```
export const widgets = pgTable('widgets', {
  id:        bigserial('id', { mode: 'number' }).primaryKey(),
  name:      text('name').notNull(),
  active:    boolean('active').notNull().default(true),
  createdAt: timestamp('created_at', { withTimezone: true })
               .notNull().defaultNow(),
})
```

Apply the schema change:

```
make db-push
```

14.1.2 Step 2: Add Shared Types

In packages/types/src/index.ts, add the interface:

```
export interface Widget {
  id:        number
  name:      string
  active:    boolean
  createdAt: string
}
```

14.1.3 Step 3: Add the Backend Route Handler

Create apps/backend/src/app/api/admin/widgets/route.ts:

```
import { NextRequest, NextResponse } from 'next/server'
import { requireRole, isAuth } from '@/lib/auth/middleware'
import { db } from '@/lib/db/client'
import { widgets } from '@/lib/db/schema'

export async function GET(req: NextRequest) {
  const session = await requireRole('root')(req)
  if (!isAuth(session)) return session

  const rows = await db.select().from(widgets).orderBy(widgets.name)
  return NextResponse.json(rows)
}

export async function POST(req: NextRequest) {
  const session = await requireRole('root')(req)
  if (!isAuth(session)) return session

  const { name } = await req.json()
  if (!name) return NextResponse.json(
    { error: 'name required' }, { status: 400 })

  const [row] = await db.insert(widgets).values({ name }).returning()
  return NextResponse.json(row, { status: 201 })
}
```

14.1.4 Step 4: Add the Frontend API Wrapper

In apps/frontend/src/lib/api.ts, the generic `get` and `post` wrappers already exist. Call them from any Server Component or client action:

```
import { get, post } from '@/lib/api'
import type { Widget } from '@open-hybrid-cloud/types'

export const fetchWidgets = (token: string) =>
  get<Widget[]>('/api/admin/widgets', token)

export const createWidget = (name: string, token: string) =>
  post<Widget>('/api/admin/widgets', { name }, token)
```

14.1.5 Step 5: Add the Frontend Page

Create apps/frontend/src/app/admin/widgets/page.tsx:

```
import { auth } from '@/lib/auth'
import { fetchWidgets } from '@/lib/api/widgets'
import { redirect } from 'next/navigation'

export default async function AdminWidgetsPage() {
  const session = await auth()
  if (!session) redirect('/login')
```

```
  const widgets = await fetchWidgets(session.apiToken ?? '')

  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-4">Widgets</h1>
      <ul>
        {widgets.map(w => (
          <li key={w.id} className="py-2 border-b">{w.name}</li>
        ))}
      </ul>
    </div>
  )
}
```

14.1.6 Step 6: Verify

```
make type-check # TypeScript must pass clean
make lint       # ESLint must pass clean
```

Navigate to http://localhost:3000/admin/widgets and confirm the page renders correctly.

Part V

Project Manager & Product Lifecycle Guide

Chapter 15

Ordering Infrastructure

15.1 The Order Lifecycle

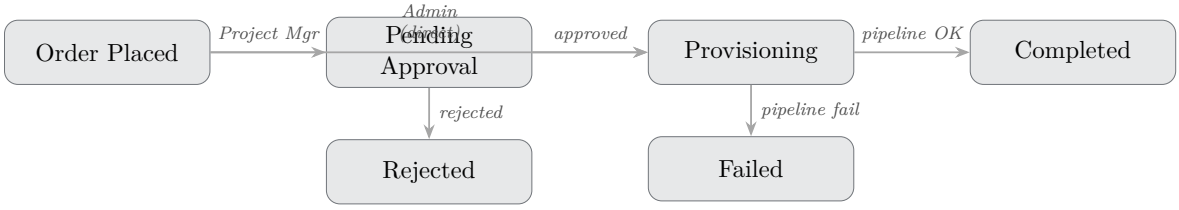

Figure 15.1: Order lifecycle state machine

15.2 Placing an Order

1. Navigate to **Catalog** and select a product.
2. Click **Configure** → select the deployment environment.
3. Select or create a **Project**.
4. Fill in the **Parameters** (required fields are marked).
5. Select or confirm the **Cost Center**.
6. Click **Deploy Now**.

As a **Project Manager** the order status changes to *Pending Approval*. An Admin receives an email notification.

As a **Admin** the GitLab webhook fires immediately.

15.3 Tracking Status

After placing an order, navigate to **My Orders → {order}**. Refresh the page to see the latest status. The backend updates the order automatically when the CI provider

sends a pipeline webhook event.

15.4 Using an Existing Deployment as a Template

1. Open **Infrastructure**.
2. Find the resource and click **Use as Template**.
3. The order form opens pre-filled with the original parameters.
4. Modify as needed and submit.

15.5 Decommissioning

1. Open **Infrastructure**.
2. Find the resource and click **Decommission**.
3. Confirm the action.
4. The destroy webhook fires; status changes to *Decommissioning*.
5. On pipeline completion, status becomes *Decommissioned*.

Cascade Decommissioning on Deletion

Deleting a **project**, **product**, or **category** automatically triggers decommissioning for all active infrastructure elements associated with that object before the database record is removed:

- **Project deleted** — all active infrastructure elements of the project receive a destroy webhook call.
- **Product deleted** — all active infrastructure elements provisioned from that product receive a destroy webhook call.
- **Category deleted** — all active infrastructure elements of every product in the category receive a destroy webhook call.

Infrastructure already in status *Decommissioning* or *Decommissioned* is skipped. Dependent database rows (translations, parameters, environment assignments) are removed automatically via PostgreSQL `ON DELETE CASCADE` constraints after the webhooks have been triggered.

Important

Deleting a project, product, or category is irreversible. All associated cloud resources will be destroyed by OpenTofu. Verify there are no active workloads before proceeding.

Chapter 16

Introducing a New Product

This chapter describes the complete process for making a new infrastructure product available in the webshop. Follow the steps sequentially.

16.1 Overview

1. Plan the product
2. Prepare the OpenTofu module
3. Configure the GitLab webhook
4. Create the product in the Admin UI
5. Define parameters (import or manual)
6. Configure deployment environments and pricing
7. Set up AI translations
8. Test the full order flow
9. Publish

16.2 Step 1: Plan the Product

Before touching any UI, answer these questions:

- **What does the product deploy?** (e.g. *Managed PostgreSQL VM on Linode*)
- **Which `infra-templates` template delivers it?** (e.g. `linode/virtual-machine`, `vsphere/virtual-machine`, or a new template)
- **In which environments is it available?** (e.g. Linode Frankfurt, vSphere on-premises)
- **What parameters does the user need to provide?** (e.g. `hostname`, `instance_type`, `region`)

- **What is the base price in EUR per environment?**
- **Who assigns the cost center?** (project / select / overhead)

16.3 Step 2: Prepare the OpenTofu Module

If the product maps to an existing template (e.g. `linode/virtual-machine`), skip to Step 3. For a new product type, add a new template to `infra-templates`:

1. Create `templates/<provider>/<name>/` in the `infra-templates` repository.
2. Add the standard OpenTofu files: `provider.tf`, `backend.tf` (empty `http` block), `variables.tf`, `main.tf`, `outputs.tf`.
3. Add `templates/<provider>/<name>/.gitlab-ci.yml` (include `.ci/base.gitlab-ci.yml`, set `TEMPLATE_DIR`).
4. Register the template in the root `.gitlab-ci.yml` dispatch block.
5. Declare all user-provided parameters as `variable` blocks in `variables.tf` with `description`, `type`, and `default` where applicable:

```
variable "db_name" {
  description = "Name of the PostgreSQL database to create."
  type        = string
}

variable "db_size_gb" {
  description = "Database volume size in GB."
  type        = number
  default     = 20
}
```

6. Test the module locally:

```
tofu init -backend-config="address=..." \
          -backend-config="username=..." \
          -backend-config="password=..."
tofu plan -var="linode_token=$TOKEN" \
          -var="db_name=test" \
          -var="db_size_gb=20"
tofu apply
tofu destroy
```

7. Commit and push to the `main` branch.

16.4 Step 3: Configure the GitLab Webhook

In the GitLab repository:

1. Go to **Settings → CI/CD → Pipeline trigger tokens**.
2. Click **Add new token** and give it a descriptive name (e.g. *webshop-prod*).

3. Note the **trigger URL** and **token**.

The trigger URL has the form:

```
https://gitlab.example.com/api/v4/projects/<PROJECT_ID>/ref/main/trigger/pipeline
```

16.5 Step 4: Create a Category (if needed)

Under **Administration → Categories**:

1. Click **New Category**.
2. Enter a **Name**.
3. Optionally assign a **Category Parameter Set** (see Step 5).
4. Save.

16.6 Step 5: Create the Product

Under **Administration → Products → New**:

16.6.1 Basic Information

1. Select the **Category**.
2. Enter the product **Name** in the primary language.
3. Enter a detailed **Description** in the primary language.
4. Upload a product **Image** (PNG or JPEG, max 10 MB).
5. Click **Save & Continue**.

16.6.2 Parameters

Choose one of two import methods:

Option A — Import from `variables.tf` (recommended):

1. Click **Browse Repository**.
2. Select the **GitLab Source** and the repository.
3. Select the branch (`main`) and navigate to `variables.tf`.
4. Click **Import Parameters**.
5. The webshop parses the HCL and creates parameter records for each variable. Review and adjust:
   - **Required** — must the user fill this in?
   - **Sensitive** — should the value be masked in the UI and audit log?

- **Default value** — pre-fill the order form.
- **Type** — text, number, boolean, or dropdown.

Option B — Manual entry:

Click **Add Parameter** and fill in each field manually.

> **Note**
>
> Parameters whose names match OpenTofu variable names (e.g. `instance_type`) are passed to the webhook as `TF_VAR_instance_type`. The name in the webshop must match the `variable` block name in `variables.tf` exactly.

16.6.3 Deployment Environments

1. Click **Add Environment**.
2. Select an environment from the dropdown (must be pre-configured under Administration → Environments).
3. Enter the **Price** in the base currency.
4. For the webhook, either:
   - Use the environment's default webhook URL, or
   - Enter a product-specific webhook URL and token.
5. Configure the **Cost Center Mode**:

   Project Cost center inherited from the project.

   Select Orderer selects from the cost center list.

   Overhead Always assigns a fixed cost center.
6. Repeat for each environment.

16.6.4 Multiple Webhooks (Pipeline Arrays)

A single product-environment combination can have multiple webhooks executed in order (e.g. first provision the VM, then configure DNS, then register in Vault). Under the webhook section:

1. Click **Add Webhook**.
2. Enter URL, token, and **Execution Order** (1, 2, 3 . . .).
3. Webhooks are triggered sequentially in ascending order.

16.7 Step 6: Configure AI Translation

1. Ensure an AI provider is configured (Chapter 7).

2. Open the product in **Administration → Products**.
3. Click **Generate AI Translation**.
4. Review the generated translations for all 25 languages.
5. Edit any that require adjustment.
6. Save.

Tip

Generate translations early in the process. The AI translates both the product name and description. Individual translations can be re-generated for specific languages by editing the translation tab directly.

16.8 Step 7: Global and Category Parameter Sets

Global parameters (under **Administration → Global Parameters**) apply to every order regardless of product. Use them for organization-wide fields such as a cost center code or project tag that must always be captured.

Category parameters (under **Administration → Categories → {category} → Parameters**) apply to all products within a category. Use them to avoid repeating parameters shared by related products (e.g. all VM products share a `region` parameter).

Parameter hierarchy (applied in this order on the order form):

1. Global parameters
2. Category parameters
3. Product parameters
4. Environment-specific parameters (override per-environment)

16.9 Step 8: Test the Full Order Flow

1. Log in as a **Admin** (Admins trigger webhooks directly).
2. Navigate to **Catalog** and find the new product.
3. Click **Configure**, fill in test parameters, and click **Deploy Now**.
4. Monitor the order under **My Orders** — the status should progress from *Provisioning* to *Completed*.
5. Verify the infrastructure was created in the Linode Cloud Manager.
6. Test the email notification (check the admin email inbox).
7. Test the **Decommission** flow:
    - Open **Infrastructure**.

- Click **Decommission** on the test resource.
- Verify the destroy pipeline runs and the resource disappears from the Linode Cloud Manager.

Important

Always test decommission in a non-production environment first. A destroy pipeline permanently deletes cloud resources and their data.

16.10 Step 9: Test with a Project Manager Account

1. Log in (or create a test account) as a **Project Manager**.
2. Place an order for the new product.
3. Verify the order shows *Pending Approval*.
4. Switch to the **Admin** account.
5. Approve the order under **Approvals**.
6. Verify the webhook fires and provisioning completes.
7. Verify the Project Manager receives the confirmation email.

16.11 Step 10: Product is Live

Once testing passes:

- The product is immediately visible in the catalog.
- No deployment or restart of the webshop is required.
- Announce the new product to users via the notification service or the shop's subtitle/branding.

Chapter 17

Project Management

17.1 Projects

A *project* is a container for related infrastructure orders. Every order must be associated with a project. Projects have:

- A **Name** and optional **Description**.
- An optional **Cost Center** (used when cost center mode is *Project*).
- An owner (the user who created it).

Project Managers see only their own projects. **Admins** and **Root users** see all projects.

17.2 Infrastructure Overview

The **Infrastructure** page shows all deployed resources grouped by project and environment. Each resource shows:

- Product name and environment.
- Deployment date and current status.
- Key parameters (e.g. VM label, cluster name).
- Buttons: **Use as Template**, **Decommission**.

17.3 Cost Center Assignment

Cost centers are maintained under **Administration → Cost Centers**. The three assignment modes give fine-grained control over who can allocate costs:

Project No user input; cost center comes from the project. Use for products where cost is always allocated to the project budget.

Select Orderer picks from the cost center list. Use when individual orders can span cost centers.

Overhead Always assigns a fixed cost center regardless of order. Use for shared services (monitoring, DNS).

Appendix A

Supported Languages

The following 25 languages are supported in the UI and for AI product translation:

Code	Language	Native name
de	German	Deutsch
en	English	English
fr	French	Français
it	Italian	Italiano
es	Spanish	Español
pt	Portuguese	Português
nl	Dutch	Nederlands
pl	Polish	Polski
cs	Czech	Čeština
sk	Slovak	Slovenčina
hu	Hungarian	Magyar
ro	Romanian	Română
bg	Bulgarian	Balgarski (Cyrillic)
hr	Croatian	Hrvatski
sl	Slovenian	Slovenščina
et	Estonian	Eesti
lv	Latvian	Latviešu
lt	Lithuanian	Lietuvių
fi	Finnish	Suomi
sv	Swedish	Svenska
da	Danish	Dansk
el	Greek	Ellinika (Greek script)
mt	Maltese	Malti
ru	Russian	Russkij (Cyrillic)
uk	Ukrainian	Ukrayinska (Cyrillic)

Table A.1: Supported language codes

Appendix B

Environment Variable Reference

Backend (`apps/backend/.env`)

Variable	Description	Required
`DATABASE_URL`	PostgreSQL DSN (`postgresql://user:pass@host/db`)	Yes
`JWT_SECRET`	HS256 signing secret — min. 32 characters	Yes
`ADMIN_EMAIL`	Email of the initial Root account	Yes
`ADMIN_PASSWORD`	Password of the initial Root account	Yes
`FRONTEND_URL`	Allowed CORS origin (default: `http://localhost:3000`)	No
`SMTP_HOST`	SMTP server hostname	No
`SMTP_PORT`	SMTP port (default: `1025`)	No
`SMTP_FROM`	Sender email address	No
`SMTP_USER`	SMTP authentication username	No
`SMTP_PASS`	SMTP authentication password	No
`SMTP_TLS`	`true` to enable SMTP TLS	No
`EXCHANGE_RATE_API_URL`	Exchange rate API endpoint URL	No
`ENTRA_TENANT_ID`	Microsoft Entra ID tenant ID (future SSO)	No
`ENTRA_CLIENT_ID`	Entra ID application (client) ID	No
`ENTRA_CLIENT_SECRET`	Entra ID client secret	No

Table B.1: Backend environment variable reference

Frontend (`apps/frontend/.env`)

Variable	Description	Required
NEXT_PUBLIC_API_URL	Backend URL reachable from the browser (used for client-side API calls)	Yes
API_URL	Backend URL reachable from the Next.js SSR server	Yes
NEXTAUTH_URL	Canonical public URL of the frontend	Yes
NEXTAUTH_SECRET	NextAuth session cookie encryption secret (min. 32 chars)	Yes

Table B.2: Frontend environment variable reference

Appendix C

OpenTofu Module Variable Reference

Variables marked *sensitive* must be set as GitLab CI/CD project or group variables with the `TF_VAR_` prefix — never as product parameters in the portal. All other variables correspond to portal product parameters (lowercased).

C.1 Module: linode-instance

Variable	Type	Description	Default
`label`	string	Unique instance label	—
`region`	string	Linode region	`eu-central`
`type`	string	Linode instance plan	`g6-nanode-1`
`image`	string	OS image slug	`linode/ubuntu22.04`
`root_pass`	string	Root password *(sensitive)*	—
`authorized_key`	string	SSH public key (leave empty to skip)	`""`
`tags`	list	Resource tags	`[]`

Table C.1: linode-instance module variables

Outputs: `id` (number), `ip_address`, `ipv6`, `label`.

C.2 Module: linode-firewall

Variable	Type	Description	Default
`label`	string	Firewall label	—
`linode_id`	number	Linode instance ID to attach the firewall	—
`inbound_ports_csv`	string	Comma-separated TCP ports to allow inbound	`"22"`

Table C.2: linode-firewall module variables

C.3 Module: linode-dns-record

Variable	Type	Description	Default
`domain_id`	number	Linode DNS domain ID	—
`hostname`	string	A record name	—
`ip_address`	string	IPv4 address for the A record	—
`ttl_sec`	number	TTL in seconds	`300`

Table C.3: linode-dns-record module variables

C.4 Module: vsphere-vm

Variable	Type	Description	Default
`name`	string	VM name in vSphere	—
`datacenter`	string	vSphere datacenter name	—
`cluster`	string	vSphere compute cluster	—
`datastore`	string	Datastore name for VM files	—
`template_name`	string	VM template to clone	—
`network`	string	Port group name	`"VM Network"`
`num_cpus`	number	vCPUs	`2`
`memory_mb`	number	Memory in MB	`2048`
`disk_size_gb`	number	Disk in GB	`40`
`ipv4_address`	string	Static IP (empty = DHCP)	`""`
`ipv4_gateway`	string	Default gateway (required if static IP set)	`""`
`vsphere_server`	string	vCenter hostname *(sensitive)*	—
`vsphere_user`	string	vCenter username *(sensitive)*	—
`vsphere_password`	string	vCenter password *(sensitive)*	—

Table C.4: vsphere-vm module variables

C.5 Template: linode/virtual-machine (Portal Product Parameters)

Parameter	Type	Description	Default
`hostname`	string	Instance label / state key	—
`region`	string	Linode region	`eu-central`
`instance_type`	string	Linode instance plan	`g6-nanode-1`
`image`	string	OS image slug	`linode/ubuntu22.04`
`inbound_ports_csv`	string	Open TCP ports (CSV)	`"22"`

Table C.5: linode/virtual-machine template product parameters

C.6 Template: vsphere/virtual-machine (Portal Product Parameters)

Parameter	Type	Description	Default
`hostname`	string	VM name / state key	—
`datacenter`	string	Datacenter name	—
`cluster`	string	Compute cluster	—
`datastore`	string	Datastore	—
`template_name`	string	VM template to clone	—
`network`	string	Port group	`"VM Network"`
`num_cpus`	number	vCPUs	`2`
`memory_mb`	number	Memory in MB	`2048`
`disk_size_gb`	number	Disk in GB	`40`
`ipv4_address`	string	Static IP (empty = DHCP)	`""`
`ipv4_gateway`	string	Gateway (required if static IP set)	`""`

Table C.6: vsphere/virtual-machine template product parameters

Appendix D

HTTP Route Reference

All backend REST endpoints are Next.js route handlers under `apps/backend/src/app/api/`. The frontend is a standard Next.js App Router application; its pages are not listed here.

The **Role** column uses the following abbreviations:

- **public** — no authentication required
- **auth** — any authenticated user (any role)
- **admin** — role `admin` or `root`
- **root** — role `root` only

Method	Path	Description	Role
Authentication & Health			
POST	`/api/auth/login`	Issue JWT from credentials	public
GET	`/api/auth/me`	Return caller profile	auth
POST	`/api/auth/logout`	Stateless no-op (client discards JWT)	auth
GET	`/api/health`	Health check + bootstrap trigger	public
Catalog			
GET	`/api/catalog`	List products by category	auth
GET	`/api/catalog/{id}`	Product detail with environments	auth
GET	`/api/catalog/{id}/image`	Serve product image (BYTEA)	auth
POST	`/api/catalog/{id}/order`	Place an order	auth
Orders			
GET	`/api/orders`	List orders (role-scoped)	auth
GET	`/api/orders/{id}`	Order detail	auth
POST	`/api/orders/{id}/approve`	Approve order and trigger pipeline	admin
POST	`/api/orders/{id}/reject`	Reject order with mandatory note	admin
Projects			
GET	`/api/projects`	List projects (role-scoped)	auth
POST	`/api/projects`	Create project	auth

Method	Path	Description	Role
GET	`/api/projects/{id}`	Project detail	auth
PUT	`/api/projects/{id}`	Update project	auth
DELETE	`/api/projects/{id}`	Delete project + cascade decommission	auth
Infrastructure			
GET	`/api/infrastructure`	Infrastructure overview (role-scoped)	auth
GET	`/api/infrastructure/{id}`	Infrastructure element detail	auth
POST	`/api/infrastructure/{id}/decommission`	Trigger destroy pipeline	auth
Admin — Users (role: root)			
GET	`/api/admin/users`	User list	root
POST	`/api/admin/users`	Create user	root
PUT	`/api/admin/users/{id}`	Update user	root
DELETE	`/api/admin/users/{id}`	Delete user	root
Admin — Catalog (role: root)			
GET	`/api/admin/categories`	Category list	root
POST	`/api/admin/categories`	Create category	root
PUT	`/api/admin/categories/{id}`	Update category	root
DELETE	`/api/admin/categories/{id}`	Delete category + cascade	root
GET	`/api/admin/products`	Product list	root
POST	`/api/admin/products`	Create product	root
GET	`/api/admin/products/{id}`	Product detail (admin view)	root
PUT	`/api/admin/products/{id}`	Update product	root
DELETE	`/api/admin/products/{id}`	Delete product + cascade	root
PUT	`/api/admin/products/{id}/image`	Upload product image	root
Admin — Environments & CI Sources (role: root)			
GET	`/api/admin/environments`	Deployment environment list	root
POST	`/api/admin/environments`	Create environment	root
PUT	`/api/admin/environments/{id}`	Update environment	root
DELETE	`/api/admin/environments/{id}`	Delete environment	root
GET	`/api/admin/ci-sources`	CI source list	root
POST	`/api/admin/ci-sources`	Create CI source	root
PUT	`/api/admin/ci-sources/{id}`	Update CI source	root
DELETE	`/api/admin/ci-sources/{id}`	Delete CI source	root
Admin — CI Repository Browser (role: root)			
GET	`/api/admin/ci/projects`	Browse CI provider projects	root
GET	`/api/admin/ci/branches`	Browse branches in a project	root
GET	`/api/admin/ci/files`	Browse files in a project	root
POST	`/api/admin/ci/import-vars`	Import vars from `variables.tf`	root
Admin — Cost Centers (role: root)			
GET	`/api/admin/cost-centers`	Cost center list	root
POST	`/api/admin/cost-centers`	Create cost center	root
PUT	`/api/admin/cost-centers/{id}`	Update cost center	root
DELETE	`/api/admin/cost-centers/{id}`	Delete cost center	root
Admin — Configuration (role: root)			
GET	`/api/admin/branding`	Get branding settings	root
PUT	`/api/admin/branding`	Update branding settings	root
GET	`/api/admin/branding/logo`	Serve branding logo (BYTEA)	root

Method	Path	Description	Role
PUT	`/api/admin/branding/logo`	Upload branding logo	root
GET	`/api/admin/config`	Get app config (SMTP, exchange)	root
PUT	`/api/admin/config/smtp`	Update SMTP config	root
PUT	`/api/admin/config/exchange`	Update exchange rate API config	root
Admin — Audit (role: admin)			
GET	`/api/admin/audit`	Paginated audit log	admin
CI Provider Webhooks (inbound, token-authenticated)			
POST	`/api/webhooks/gitlab/pipeline`	GitLab pipeline status event	public
POST	`/api/webhooks/github/pipeline`	GitHub Actions workflow event	public
POST	`/api/webhooks/bitbucket/pipeline`	Bitbucket pipeline status event	public

Table D.1: Complete REST API route reference

Appendix E

Requirements Traceability Matrix

The following table maps each functional and non-functional requirement to the implementation location and the handbook section that describes it.

ID	Requirement	Implementation	Section
FA-01	User authentication (local credentials + JWT)	apps/backend/src/app/api/auth/login/route.ts, apps/backend/src/lib/auth/	§8
FA-02	Role-based access control (admin, root, project_manager)	apps/backend/src/lib/auth/middleware.ts	§8
FA-03	Product catalog with categories	apps/backend/src/app/api/catalog/	§15
FA-04	Configurable product parameters	apps/backend/src/lib/db/schema.ts (parameters table)	§16.6
FA-05	Order placement flow	apps/backend/src/app/api/catalog/[id]/order/route.ts	§15
FA-06	Order approval workflow	apps/backend/src/app/api/orders/[id]/approve/route.ts	§15
FA-07	CI provider webhook trigger (GitLab / GitHub / Bitbucket)	apps/backend/src/lib/ci/index.ts	§4.7
FA-08	Inbound pipeline status via CI webhooks (no polling)	apps/backend/src/app/api/webhooks/[provider]/pipeline/route.ts, apps/backend/src/lib/webhook/handler.ts	§4.7
FA-09	Infrastructure overview	apps/backend/src/app/api/infrastructure/	§17.2
FA-10	Decommission (destroy) flow	apps/backend/src/app/api/infrastructure/[id]/decommission/route.ts	§15.5
FA-09.5–9.8	Cascade decommission on project/product/category delete	apps/backend/src/app/api/projects/[id]/route.ts	§15.5
FA-11	Audit log	apps/backend/src/app/api/admin/audit/route.ts	§9
FA-13	Multi-currency support	apps/backend/src/lib/db/schema.ts (exchange_rates), apps/backend/src/app/api/admin/config/exchange/	§7.6
FA-14	Branding configuration	apps/backend/src/app/api/admin/branding/	§7.5
NFA-01.1	Docker deployment (two containers)	apps/backend/Dockerfile, apps/frontend/Dockerfile, infra/docker-compose.yml	§6.3
NFA-01.2	Kubernetes deployment	infra/helm/open-hybrid-cloud/	§6.4
NFA-01.3	Schema management via Drizzle ORM	apps/backend/src/lib/db/schema.ts, make db-push	§3
NFA-02.1	HTTPS enforcement	Nginx config (reverse proxy, TLS termination)	§6.3
NFA-02.2	JWT authentication (HS256, 24 h)	apps/backend/src/lib/auth/jwt.ts, jose library	§3
NFA-02.3	bcrypt password hashing	apps/backend/src/app/api/auth/login/route.ts, bcryptjs	§3
NFA-02.4	Role-enforced API routes	apps/backend/src/lib/auth/middleware.ts	§8
NFA-04	Audit trail for all state changes	apps/backend/src/lib/db/schema.ts (audit_log)	§9

ID	Requirement	Implementation	Section

Table E.1: Requirements traceability matrix

Appendix F

Compiling This Handbook

The handbook source is at `docs/handbook.tex`. Compile with:

```
# From the repo root
make docs

# Or manually (run twice for ToC/references)
cd docs
pdflatex handbook.tex
pdflatex handbook.tex
```

Required LaTeX packages: `lmodern`, `geometry`, `fancyhdr`, `xcolor`, `tikz`, `pgf`, `booktabs`, `longtable`, `tabularx`, `listings`, `tcolorbox`, `hyperref`, `enumitem`, `float`, `caption`, `graphicx`, `microtype`, `parskip`, `pdflscape`, `pifont`.

All packages are available in TeX Live 2023+ and MiKTeX 23+.